

Maio de 2012



TUTORIAL DE PASCAL

TUTORIAL DE PASCAL
Igor Nunes (@*thoga31*), Comunidade Portugal-a-Programar®
Maio de 2012

O presente livro é um tutorial de Pascal com os devidos direitos de autor.

Autor: Igor C. B. Nunes

Nick no Portugal-a-Programar®: thoga31

Última revisão: Maio de 2012

Igor Nunes é Moderador Local nos quadros de Pascal e Delphi/Lazarus no Portugal-a-Programar®, bem como é membro da *Wiki Team*.

Este livro não deve ser utilizado para outros fins se não a auto-aprendizagem sem o devido consentimento do autor.

Caso pretendam saber mais informações, por favor contactem-me por PM no P@P.

Maio de 2012

TUTORIAL DE PASCAL

Revisto e actualizado

Igor Nunes (*thoga31*)

P@P[®]

O presente tutorial é a versão PDF do original online, que pode ser consultado no
Fórum da Comunidade Portugal-a-Programar[®], seguindo o link:

<http://www.portugal-a-programar.pt/topic/41246-pascal-tutorial-de-pascal-2011/>

Publicado a 17 de Fevereiro de 2011.



thoga 31

Índice

Nota Introdutória do Autor	6
Objectivos Gerais do Tutorial	7

Parte I - Preliminares. Programação básica. 8

1. Breve história, nível da linguagem e compiladores	8
2. Variáveis e tipos de dados	8
3. Operações e operadores	9
3.1. Igualdade, Diferença e Negação	9
3.2. Soma, subtracção, multiplicação e divisão	9
3.3. Gravar um resultado numa variável – atribuição	9
3.4. Valor absoluto e truncado, logaritmo, seno, co-seno e tangente, número ao quadrado e raiz quadrada	9
4. Declarações e noções sobre Pascal	10
4.1. Declarações iniciais e de variáveis e enunciações básicas	10
4.2. Atribuir valores num array	11
4.3. Escrita de textos e leitura de dados	12
5. Estruturas de Decisão e de Repetição	12
5.1. Condição If... Then... Else...	12
5.2. Ciclo Repeat... Until...	13
5.3. Ciclo While... Do...	13
5.4. Ciclo For... to/downto... do...	14
5.5. Condição Case of... Else...	14
6. Procedimentos e funções	15
6.1. Procedimentos	15
6.2. Funções	15
7. Arredondamentos	16

Parte II - Componente Prática para Aplicação de Conhecimentos. 17

1. <i>Hello World!</i>	17
2. A primeira calculadora – uso do If... Then... Else...	17
3. Avaliações 1 – uso do Case... Of...	19
4. Avaliações 2 – uso do Repeat... Until...	20
5. Ordenação crescente de uma lista – uso do ciclo For... To... Do...	21
6. Contagem de espaços e palavras numa frase – uso de string	23
7. Cálculo da tangente – uso de uma função	24

Parte III - Funções e Procedimentos padrão. Conversão. Personalização gráfica. 26

1. Operações numéricas gerais	26
2. Optimização básica	27
3. Facilitação da leitura de um programa	28
4. A fórmula resolvente – programa completo – uso de funções	29

5. Conversão de variáveis	30
5.1. Real/Integer – String	30
5.2. Real – Integer	31
6. Coloração de texto e fundo	31
7. Caracteres ASCII	32

Parte IV - Registos. Ficheiros de texto simples. 34

1. A variável Record	34
1.1. A cláusula With	36
2. Códigos úteis sobre o teclado e o cursor	36
2.1. Posição do cursor	36
2.2. Determinação da tecla premida	37
3. A variável Text – escrita e leitura de ficheiros de texto simples – parte 1	37
4. Códigos úteis sobre string	38
5. Aleatoriedade	39

Parte V - Conjuntos. Estruturação de um programa em Pascal. Data e Hora do Sistema. 41

1. Conjuntos	41
1.1. Declaração e utilização	41
1.2. Operações com conjuntos	43
1.3. Comparação de conjuntos	45
1.4. Análise de uma linha de texto	45
2. Estruturação de um programa em Pascal	46
2.1. Estrutura Begin... End	47
2.2. A cláusula Exit	47
2.3. Etiquetas	48
2.4. Características desejáveis num programa	49
3. Data e hora do sistema	49

Parte VI - Recursividade. Variáveis de Texto Simples. Estrutura de tentativa. Lista padrão do Pascal. 51

1. Recursividade	51
2. Variáveis Text – parte 2	52
3. Sucessor e predecessor	54
4. Estrutura de tentativa – Try... Except... Finally...	54
5. Lista padrão do Pascal	56
5.1. Palavras reservadas	56
5.2. Funções e identificadores padrão	57
5.3. Procedimentos padrão	57
5.4. Operadores	57
5.5. Tipos de dados	57
5.6. Tabela ASCII	58

Parte VII – Hexadecimais, Binários e Numeração Romana. Message Box.

	59
1. Breve introdução aos Hexadecimais e Binários	59
2. Estrutura de tentativa – mensagem exacta do erro	59
3. Título da consola	60
4. Message Box	61
4.1. Criação auxiliada no Dev-Pascal 1.9.2	61
5. Numeração Romana	63
Notas Finais	64
Propostas de Resolução de Exercícios e Desafios seleccionados	65
Exercício 1	65
Exercício 2	65
Exercício 3	66
Exercício 4	66
Exercício 5	67
Exercício 6	67
Exercício 7	68
Exercício 8	68
Exercício 9	69
Exercício 11	70
Desafio 1	71
Desafio 2	71
Desafio 3	72
Desafio 4	72

Nota Introdutória do Autor

É com enorme gosto que tenho vindo a escrever o presente Tutorial, iniciado em Junho de 2010, e agora edito esta versão totalmente revista e actualizada.

O acto de programar deverá ser visto, penso eu, como uma **arte**, pois tal como não é qualquer um que faz uma escultura greco-romana, também não é qualquer um que pega num problema do quotidiano, ou não, e desenvolve praticamente do nada uma solução consistente, eficiente e com o menor nível de erro possível. Programar **bem** é uma arte que, apesar de muitas vezes vista como a área dos “burrinhos que mais nada sabem do que ter um monitor à frente e jogar”, é a arte que permite que essas mesmas pessoas possam desenvolver os seus trabalhos pessoais, os seus projectos, ter tudo muito simplificado graças a um computador e a um programa realizado por um ou mais **programadores**. Afinal, os programadores são mesmo uns “burrinhos”.

Muito se discute, também, sobre qual a melhor linguagem de programação para se iniciar. E eu respondo: depende. Depende dos vossos objectivos, depende da vossa fome de aprender, depende dos vossos projectos que têm no vosso horizonte.

Em muitas escolas, faculdades e/ou universidades, institutos politécnicos, entre outros, dá-se o Pascal como a primeira linguagem por ser estruturada, geralmente intuitiva e, actualmente, com os horizontes abertos para os novos paradigmas.

O presente Tutorial não dispensa totalmente a consulta de bibliografia extra, pois a sua quase totalidade foi feita sem recurso a qualquer uma. Poderão haver designações técnicas diferentes das que aqui são ditas, pois, de quando em vez, utilizo antes as minhas próprias expressões por achar que são mais intuitivas.

Este Tutorial tenta quebrar certos padrões estruturais do ensino desta linguagem de programação. É um método totalmente pessoal, em que haverá quem goste e quem não goste, claro.

Por último, boa viagem no mundo Pascal.

O autor,
Igor Nunes
(@thoga31)

Objectivos Gerais do Tutorial

O presente Tutorial direcciona-se essencialmente a principiantes em Pascal. Tem por objectivo divulgar o Pascal como uma linguagem com grandes potencialidades para a aprendizagem de Programação, pois a sua linguagem aproxima-se deveras ao inglês.

De início são abordados os preliminares da linguagem, incluindo uma breve referência histórica, sob uma forte componente teórica, que é consolidada na Parte 2, só de componente prática. Seguem-se partes teórico-práticas sobre as estruturas mais importantes do Pascal, num total de quatro partes.

Cada Parte tem uma breve introdução para dar a conhecer as focalizações desta, bem como inclui um Índice para orientação inicial sobre os temas tratados e a sua organização.

Por último, repetindo, pessoalmente falando, o que já disse no tópico original... *Enjoy it!*

PARTE I

Preliminares. Programação básica.

Aqui começam por ser dados os grandes alicerces essenciais para se programar em Pascal. As estruturas essenciais, as principais palavras reservadas, a estruturação básica de um programa em Pascal. Esta é uma parte totalmente teórica, que terá a sua consolidação na Parte II, só de aplicações práticas dos assuntos agora tratados.

1. Breve história, nível da linguagem e compiladores

Após a criação do Assembly como primeira linguagem de programação que não exigia que o programador escrevesse os seus programas directamente em código binário, Niklaus Wirth criou, em 1971, a primeira linguagem de programação estruturada à qual, em homenagem ao grande matemático Blaise Pascal, deu o nome de **Pascal**. Esta foi uma linguagem baseada originalmente no ALGOL-60.

Niklaus Wirth preocupou-se em aproximar a linguagem de programação da linguagem do ser humano, em particular, claro está, o inglês. Depois do seu sucesso com o lançamento do Pascal, hoje em dia discute-se se esta é uma linguagem de baixo nível, médio nível ou alto nível. Se considerarmos que estas denominações foram criadas baseadas na proximidade da linguagem de programação à linguagem humana, constatamos que o Pascal é de **alto nível**. Contudo, actualmente confunde-se estas designações com as potencialidades que a linguagem oferece, e, devido a isto, muitos dizem que Pascal é de **baixo nível**.

Ora, um dos compiladores mais antigos e dos mais famosos do mundo é o *Turbo Pascal*. Contudo, mais compiladores e IDEs têm saído, muitos na tentativa de criar um compilador/IDE com um ambiente gráfico muito mais agradável que o *Turbo Pascal*, como é o caso do *Dev-Pascal* (IDE que utiliza o *GNU Pascal* ou o *Free Pascal* como compilador subjacente). O compilador mais utilizado é o primeiro referido, havendo um compilador baseado neste e também muito utilizado, que é o *Free Pascal*. Qualquer um destes IDEs/compiladores é muito bom, apesar de muitas divergências quanto a este tema. Existem compiladores que aceitam operadores da linguagem C, tais como: **+=** **-=** ***=** **/=**.

Escolhido um IDE/compilador, e partindo do pressuposto que sabem as bases sobre Lógica e Algoritmos, passamos então a referir os tipos de variáveis com que mais se trabalha, os principais operadores e afins, condições e ciclos, bem como as principais declarações do pascal, os procedimentos e as funções.

2. Variáveis e tipos de dados

Os principais tipos de dados são os seguintes:

- **Integer** – número inteiro
- **Real** – número real
- **String** – texto
- **Char** – carácter
- **Word** – palavra – representa um número inteiro, só que organizado de forma diferente em memória*
- **Longint** – número inteiro mais longo
- **Array** – matriz de valores, de 1, 2 ou mais dimensões
- **Boolean** – booleano: verdadeiro ou falso
- **Text** – ficheiro de texto simples

* - A explicação está fora do âmbito do presente Tutorial.

Nenhuma variável poderá ter o nome de um destes tipos de dados, ou seja, por exemplo, nunca uma variável de número inteiro poderá ser chamada de **Array**, pois esta é uma **palavra reservada*** do Pascal.

* - Palavra com uma função bem definida, que não pode ser redefinida. Apresenta-se uma lista na Parte VI.

3. Operações e operadores

3.1. Igualdade, Diferença e Negação

A igualdade e a diferença são úteis essencialmente em condições. A resposta que estes operadores devolvem ao programa é do tipo booleana: verdadeiro ou falso.

Por exemplo, no caso de $2 = 2$ será devolvido `True` (verdadeiro), pois, de facto, 2 é igual ($=$) a 2.

Já no caso de $2 <> 2$ será devolvido `False` (falso), pois 2 **não** é diferente de 2.

Já no caso de ser escrito `NOT (2 <> 2)`, será devolvido que é verdade, pois, de facto, 2 **não** é diferente de 2.

- `=` – igual a
- `<>` – diferente de
- `not` – não

3.2. Soma, subtracção, multiplicação e divisão

Se declararmos as variáveis A e B como números inteiros (`Integer`), então estas operações serão dadas segundo a seguinte lista:

- Soma: $A + B$
- Subtracção: $A - B$
- Multiplicação: $A * B$
- Divisão: A / B

Contudo, no caso da divisão, o Pascal tem dois operadores que devolvem um o resultado da divisão como um número inteiro e outro o primeiro resto da divisão:

- Resultado inteiro: $A \text{ div } B$
- Primeiro resto: $A \text{ mod } B$

3.3. Gravar um resultado numa variável – atribuição

O resultado destas últimas operações pode ser gravado numa variável (atribuído a uma variável) para depois ser utilizado noutros processos e cálculos. Em Pascal, o comando (operador) que dá ordem para gravar (atribuir) um valor numa variável é `:=`. Este processo é denominado de **atribuição**. Por exemplo, se quisermos somar duas variáveis inteiras, A e B, e atribuir o resultado a uma variável inteira chamada resultado, será escrito `resultado := A+B`. As variáveis `array` têm uma forma diferente de serem atribuídas, apesar de utilizarem este operador. Tal será visto mais à frente.

3.4. Valor absoluto e truncado, logaritmo, seno, co-seno e tangente, número ao quadrado e raiz quadrada

Continuando com a variável A do tipo `Integer`:

- Valor absoluto: `abs (A)`
- Valor truncado: `trunc (A)`
- Logaritmo neperiano (de base e): `ln (A)`
- Seno (ângulo em radianos, só): `sin (A)`
- Co-seno (ângulo em radianos, só): `cos (A)`
- Número ao quadrado: `sqr (A)`
- Raiz quadrada: `sqrt (A)`

Ora, pela fórmula da tangente, esta é calculada por $\sin(A) / \cos(A)$. É claro que, se o co-seno valer zero então o programa dará erro e irá abaixo. Mais à frente, nas condições, veremos como transpor esta situação.

4. Declarações e noções sobre Pascal

4.1. Declarações iniciais e de variáveis e enunciações básicas

Um programa em Pascal tem de se iniciar com as seguintes declarações:

- Nome do programa
- Bibliotecas a utilizar
- Variáveis, tipos e constantes

O nome do programa é enunciado com a palavra reservada `program`, que deverá ter o mesmo nome do ficheiro que contém o **código-fonte**, tirando a extensão do ficheiro (**`*.pas`**, **`*.pp`** ou outra).

As bibliotecas são conjuntos de operações e funções que, sem estas estarem enunciadas, não serão reconhecidas pelo compilador. Por exemplo, o comando `gettime`, que obtém a hora do computador, só funciona com a biblioteca `dos`. As bibliotecas a utilizar são declaradas com `uses`. A biblioteca mais utilizada e que contém as funções básicas é a biblioteca `crt` – esta é a biblioteca obrigatória para quase todos os programas.

Por fim, as variáveis são declaradas pela palavra reservada `var`. Contudo, para além destas, existem os Tipos de Variável e as Constantes.

As **constantes** são valores que nunca são alterados ao longo do programa. Podem ser de qualquer tipo, sendo o mais comum número real, como por exemplo o valor de `pi`. As constantes são enunciadas com `const`.

Já no caso de um conjunto de variáveis for todo do mesmo tipo (que não seja um tipo de dado simples predefinido), pode-se criar um **tipo de variável** com um nome dado por nós, para além dos enunciados anteriormente. Os novos tipos criados pelo programador são declarados por `type`.

Por exemplo:

```
program exemplo;
uses crt;
type lista = array[1..10] of integer;
var avaliacoes:lista;
    n:string;
const pi=3.14;
```

Como pode ser visto por este exemplo, **uma instrução é sempre seguida de ponto e vírgula**, excepto nalguns casos, dos quais um deles será abordado mais à frente, que é com a palavra reservada `else`. Para além disso, um tipo de variável é definido por `nome_do_tipo = tipo_de_dado`. Neste caso, foi declarado que `lista` é do tipo `array` de 1 dimensão, constituído por dez elementos, que começa no elemento nº 1 e acaba no elemento nº 10, e este `array` é feito de variáveis `integer`.

As constantes também são enunciadas como uma igualdade. Sempre que no programa for escrito numa instrução, segundo o exemplo dado, `pi`, o programa irá assumir o valor 3.14. Caso se tente atribuir um resultado qualquer a `pi`, o compilador dará erro na compilação.

As variáveis são declaradas segundo a estrutura `var nome_variavel : tipo_variavel;`. Neste caso, foi enunciado que a variável `n` é do tipo `String`.

Os **arrays**, como matrizes de variáveis, têm de ser declaradas quanto à sua dimensão* e ao tipo de variáveis que comporta: `var nome : array [dimensão] of tipo_variavel;`. A dimensão pode ser uni ou multidimensionais, ou seja, uma simples lista ou uma espécie de tabela de valores.

Por exemplo:

```
var list_numeros : array [0..5] of real;
    tabela : array [0..5,0..3] of real;
```

* - mais propriamente, é o conjunto do `array`. Contudo, tal conceito só será abordado na Parte V.

A variável `list_numeros` pode ser apresentada aos nossos olhos da seguinte forma:

0
1
2
3
4
5

Já a variável `tabela` pode ser apresentada desta forma:

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3
4,0	4,1	4,2	4,3
5,0	5,1	5,2	5,3

Por fim, após a declaração do programa, das bibliotecas, dos tipos de variáveis e/ou das variáveis e/ou das constantes, para o programa ser inicializado e finalizado resta-nos o seguinte:

```

program exemplo; // declarações iniciais
uses crt;
var A:integer;

begin // INÍCIO

// programa principal

end. // FIM

```

4.2. Atribuir valores num **array**

Continuando com as variáveis `list_numero` e `tabela`, vamos atribuir valores dentro de cada um destes arrays.

Na `list_numero`, vamos atribuir um valor na localização 2, neste caso `-4.5`:

```
list_numero[2] := -4.5
```

Assim, considerando que os restantes elementos do array ainda não foram alterados, o array `list_numero` terá o seguinte aspecto gráfico (quando uma variável é declarada, e esta é do tipo numérico, o seu valor inicial é, por defeito, 0 (zero)):

0
0
-4.5
0
0
0

Já para atribuir este valor em `tabela` na posição `(3,1)`:

```
tabela[3,1] := -4.5
```

O aspecto final do array será:

0	0	0	0
0	0	0	0
0	0	0	0
0	-4.5	0	0
0	0	0	0
0	0	0	0

4.3. Escrita de textos e leitura de dados

O programa pode escrever textos e receber dados, como é óbvio – realiza operações de *input* e de *output*. Para escrever um texto e fazer parágrafo:

```
writeln('Texto');
```

Contudo, se quisermos escrever um texto e deixar o cursor a piscar à frente deste, escrevemos:

```
write('Texto ');
```

Para ler dados:

```
readln(variável);
```

A variável lida tem de coincidir com o seu tipo. Por exemplo, se a variável for a **A**, do tipo `Integer`, faremos:

```
readln(A);
```

No caso de introduzirmos um número real ou um texto em vez de um número inteiro, o programa irá abaixo. Por isso há que avisar o utilizador desse facto:

```
write('Introduza um numero inteiro: ');
readln(A);
```

5. Estruturas de Decisão e de Repetição

Estruturas de decisão são instruções estruturadas que permitem ao programa (computador) analisar uma determinada **condição** e, consoante esta seja **verdadeira** ou **falsa**, irá **decidir** o que realizar de seguida. Doravante, nesta parte, para simplificar, serão designadas por **condições**, que é, de facto, uma outra designação bastante comum e muito utilizada mesmo entre profissionais.

Já as **estruturas de repetição** são instruções estruturadas que obrigam o programa repetir um conjunto de instruções conforme a **condição** é **verdadeira** ou **falsa**. Doravante, serão designadas por **ciclos**.

5.1. Condição **If... Then... Else...**

Esta condição dá uma ordem ao programa do tipo **Se... Então... Senão...**:

```
if (condição) then begin
  (comandos1)
end
else begin
  (comandos2)
end;
```

Ou seja, **se** a condição for verdadeira, **então** são executados os comandos1, **senão** são executados os comandos2.

Por exemplo, se a variável *A*, do tipo *Real*, for positiva ou nula, mostra-se o texto "numero positivo ou nulo", caso contrário mostra-se o texto "numero negativo":

```
write('Introduza um numero real: ');
readln(A);
if (A>=0) then begin
    writeln('Numero positivo ou nulo');
end
else begin
    writeln('Numero negativo');
end;
```

5.2. Ciclo Repeat... Until...

Este ciclo obriga o programa a executar a mesma série de instruções **até que** a condição imposta seja satisfeita (verdadeira).

Por exemplo, se quisermos que o utilizador introduza um número inteiro positivo e não nulo:

```
repeat // Repetir...
    readln(A);
until (A>0); // ATÉ QUE (A>0)
```

5.3. Ciclo While... Do...

Este ciclo faz o mesmo que o ciclo Repeat... Until..., contudo fá-lo com a lógica contrária. Enquanto o ciclo anterior dizia **Repetir até que a condição seja verdadeira**, este ciclo diz **Repetir enquanto a condição for verdadeira**. Recorrendo ao mesmo exemplo anterior, mas aplicando este ciclo:

```
while (A<=0) do begin // ENQUANTO (A <= 0)...
    readln(A);
end;
```

De forma a ser mais intuitivo quando se lê o código, pode-se escrever:

```
while NOT (A>0) do begin // Enquanto A não é maior que zero...
    readln(A);
end;
```

Todavia, é necessário um cuidado extra com este ciclo. Enquanto o ciclo Repeat... Until... é executado pelo menos uma vez, pois a condição imposta só é analisada após uma **iteração**^{*}, no ciclo While... a condição é a primeira coisa a ser analisada. Caso esteja de imediato satisfeita, nenhuma iteração é realizada. Para evitar esta falha, deve-se "reiniciar" a(s) variável/eis em questão:

```
A := -1; // A é, desde já, um número negativo, o que NÃO satisfaz a condição do ciclo
while NOT (A>0) do begin // Enquanto A não é maior que zero...
    readln(A);
end;
```

^{*} - Nome dado a cada vez que o bloco de instruções dentro de uma estrutura de repetição é processado.

5.4. Ciclo **For... To/Downto... Do...**

Este é um ciclo que é muito útil, por exemplo, para escrever o conteúdo de um array. Voltando ao array anteriormente descrito, tabela, consideremos que o programa já lhe atribuiu valores – para escrever o seu conteúdo no ecrã:

```
program exemplo;
uses crt;
var tabela:array[0..5,0..3] of real;
    i,j:integer;
begin
    // comandos anteriores, quaisquer, que atribuíram valores no array
    for i:=0 to 5 do begin
        for j:=0 to 3 do begin
            write(tabela[i,j],'; ');
        end;
        writeln;
    end;
end.
```

É incrementado à variável *i* (contador) o valor 1 após o conjunto de comandos ter sido realizado, assim como à variável *j*. O conjunto de comandos que está dentro do primeiro ciclo for só termina quando *i* assumir o valor 5. Assim acontece com o segundo ciclo: só termina quando *j* valer 3.

Contudo, poder-se-á escrever a ordem da tabela ao contrário. Em vez de começarmos pelo valor [0,0] e terminarmos no valor [5,3], começamos em [5,3] e acabamos em [0,0]. Em vez de escrevermos, no ciclo, *to*, escrevemos *downto*, que significa que se incrementa o valor -1 à variável contador do ciclo (a variável diminui em uma unidade):

```
program exemplo;
uses crt;
var tabela:array[0..5,0..3] of real;
    i,j:integer;
begin
    // comandos anteriores, quaisquer, que gravaram valores no array
    for i:=5 downto 0 do begin
        for j:=3 downto 0 do begin
            write(tabela[i,j]);
        end;
        writeln;
    end;
    readln;
    // impede que o programa feche logo com o END final, esperando que seja premido ENTER
end.
```

Mais à frente, na Parte II, é dada uma excelente exemplificação da utilização deste ciclo.

5.5. Condição **Case of... else**

Esta condição pode poupar a utilização excessiva de condições *if*. Esta instrução diz-nos **Caso a variável assuma este valor, então....** Se se pretender classificar uma nota de 0 a 20 em termos qualitativos, ter-se-á, por exemplo, o seguinte:

```
program avaliacoes;
uses crt;
var nota:integer;

begin
    write('Introduza uma nota, de 0 a 20: ');
    readln(nota);
    case nota of
        0..9:writeln('Insuficiente');
        10..14:writeln('Suficiente');
        15..17:writeln('Bom');
```

```

        18..20:writeln('Excelente');
    End
else begin
    if (nota<0) then writeln('ERRO! Nota nao pode ser negativa!');
    if (nota>20) then writeln('ERRO! Nota nao pode ser maior que 20!');
end;
readln;
end.

```

Caso mais do que uma instrução tenha que ser executada em determinado **caso**, a estrutura será:

```

case nota of
    0..9:begin
        writeln('Insuficiente. ');
        writeln('Chumbou! ');
        end;
    10..14: // etc...
end;

```

Na Parte II voltará a ser abordado este exemplo, estendido.

6. Procedimentos e funções

6.1. Procedimentos

Dentro de um programa pode existir um **subprograma**. Estes subprogramas são designados, em Pascal, por **procedimentos**, e são declarados antes do bloco principal do programa pela expressão reservada `procedure`.

Por exemplo, podemos criar um procedimento para calcular a raiz quadrada de um número, sem que tal seja feito no bloco principal:

```

program raiz;
uses crt;
var a,resultado:real;

procedure calc_raiz;
begin
    resultado:=SQR(a);
end;

begin
    write('Introduza valor A: ');
    readln(a);
    calc_raiz; // chama o procedimento
    writeln('A raiz quadrada de A e: ',resultado);
    readln;
end.

```

6.2. Funções

O mesmo pode ser feito através de uma **função**. Podemos, por exemplo, criar uma função, num programa, que calcule a tangente de um número sem ser necessário calcular no meio do bloco principal todas as contas necessárias. Uma função é declarada antes do bloco principal através da palavra reservada `function`. Dever-se-á ter em conta que **uma função devolve um valor** e um procedimento não, ou seja, uma função é da mesma hierarquia das funções padrão do Pascal, como `abs`, e os procedimentos são subprogramas dentro do programa principal que executam instruções sem devolverem absolutamente nada no fim. Veja-se o exemplo:

```
program tang_ang;
uses crt;
var angulo:real;

procedure ler_angulo;
begin
    write('Introduza angulo em radianos: ');
    readln(angulo);
end;

function tangente(a:real):real;
begin
    tangente:=SIN(a)/COS(a);
end;

begin
    ler_angulo; // procedimento que vai ler o ângulo, mas nada devolve.
    writeln('A tangente do angulo e: ',tangente(angulo)); // função que calcula a tangente e
    // devolve o seu resultado.
    readln;
end.
```

7. Arredondamentos

Em Pascal é possível, claro está, arredondar valores. Existem dois métodos distintos. Através da função padrão `round`, que arredonda só às unidades e pode ser atribuído o resultado a uma variável, ou através de um código que utiliza dois pontos (:).

Por exemplo, para arredondar um resultado do tipo real (sendo este a variável `resultado`) às unidades:

```
var i : integer;
    resultado : real;
// ...
i := round(resultado);
```

Com o método dos dois pontos, podemos indicar o espaço que o número ocupa e o arredondamento que tem, do género: **número:espaço:arredondamento**. Por exemplo:

```
write(resultado:12:5);
```

Neste caso, se o resultado for 34.56356976356, irá aparecer algo como (sendo “-” indicação de um espaço vazio): ----34.56357

Já se for escrito:

```
write(resultado:0:2);
```

Irá aparecer exactamente como: 34.56

Atenção!: este método só serve para a **escrita** com os procedimentos padrão `write` e `writeln`.

PARTE II

Componente Prática para Aplicação de Conhecimentos

Nesta continuação será dada ênfase à parte prática da programação em Pascal. Serão colocados igualmente Propostas de Exercícios. Esta parte do Tutorial não terá constantes explicações dos códigos para poder exercitar a leitura lógica de um programa em Pascal.

1. Hello World!

Como em qualquer linguagem, onde se começa regra geral com o famoso “Olá Mundo!”, ir-se-á realizá-lo em Pascal.

Primeiramente, há que saber que, neste programa, apenas será escrita uma mensagem, pelo que bibliotecas não são necessárias. Esta parte é, então, omitida nas **declarações iniciais**:

```
program hello_world; // Declara-se o programa, no geral.
// bibliotecas não são necessárias
// não são necessárias variáveis

begin
    writeln('Hello World!'); // escreve o texto "Hello World!"
    readln;
    // sem isto, o programa é encerrado repentinamente: é necessário pressionar ENTER.
end.
```

“Embeleze-se” o programa, onde já se irá pedir o nome do utilizador e dar uma mensagem final.

```
program hello_world; // Declara-se o programa, no geral.
uses crt;
// dependendo do IDE/compilador, esta biblioteca será necessária ou não neste exemplo
var nome:string;

begin
    write('Introduza o seu nome: ');
    // escreve o texto, e o cursor fica à frente deste, pois falta o sufixo LN (LiNe)
    readln(nome);
    writeln; // faz mais um parágrafo sem escrever texto
    writeln('Hello World!');
    writeln('Seja bem-vindo, Sr.(a) ',nome);
    // escreve o texto, seguido do conteúdo da variável NOME
    readln;
end.
```

2. A primeira calculadora – uso do If... Then... Else...

Crie-se a primeira calculadora. O utilizador **não** vai escolher o operador. Simplesmente o utilizador introduz os dois valores e aparecem os resultados das quatro operações básicas.

Necessitar-se-á de uma condição **If... Then... Else...** por causa da divisão. Se o denominador for 0 (zero), o programa, ao tentar fazer a conta, irá abaixo: a condição vai evitar este *bug*.

```
program primeira_calculadora;
uses crt;
var num1,num2:real;
```

Ou seja, utilizar-se-á duas variáveis **reais** que alojarão os dois valores necessários.

```
program primeira_calculadora;
uses crt;
var num1,num2:real;

begin
  writeln('CALCULADORA SIMPLES');
  write('Primeiro valor: ');
  readln(num1);
  write('Segundo valor: ');
  readln(num2);
end.
```

Neste momento, o utilizador é convidado a introduzir os dois valores que farão parte das quatro operações. Resta, então, realizar os cálculos e mostrá-los ao utilizador:

```
program primeira_calculadora;
uses crt;
var num1,num2:real;

begin
  writeln('CALCULADORA SIMPLES');
  write('Primeiro valor: ');
  readln(num1);
  write('Segundo valor: ');
  readln(num2);
  writeln;
  writeln(num1:0:3,' + ',num2:0:3,' = ',num1+num2:0:3);
  writeln(num1:0:3,' - ',num2:0:3,' = ',num1-num2:0:3);
  writeln(num1:0:3,' * ',num2:0:3,' = ',num1*num2:0:3);
end.
```

Contudo, chega a fase de antever a situação de $\text{num2} = 0$ por causa da divisão. Criar-se-á, então, a condição à parte.

```
write(num1:0:3,' / ',num2:0:3,' = ');
// não faz parágrafo, à frente ficará o resultado ou uma mensagem de erro
if (num2 = 0) then begin // SE num2=0 ENTÃO
  write('Infinito'); // escrever "INFINITO"
end
else begin // SE NÃO
  writeln(num1/num2:0:3); // escrever resultado
end;
```

Inclua-se no código do programa:

```
program primeira_calculadora;
uses crt;
var num1,num2:real;

begin
  writeln('CALCULADORA SIMPLES');
  write('Primeiro valor: ');
  readln(num1);
  write('Segundo valor: ');
  readln(num2);
  writeln;
  writeln(num1:0:3,' + ',num2:0:3,' = ',num1+num2:0:3);
  //a cláusula ":0:3" dá duas indicações: ocupar o espaço mínimo de 0 caracteres,
  //arredondando o número a 3 casas decimais
  writeln(num1:0:3,' - ',num2:0:3,' = ',num1-num2:0:3);
  writeln(num1:0:3,' * ',num2:0:3,' = ',num1*num2:0:3);
  write(num1:0:3,' / ',num2:0:3,' = ');
  if (num2 = 0) then begin // SE num2=0 ENTÃO
```

```

        writeln('Infinito'); // escrever "INFINITO"
    end
else begin // SE NÃO
    writeln(num1/num2:0:3); // escrever resultado
end;
readln; // esperar pelo ENTER
end.

```

3. Avaliações 1 – uso do Case... Of...

Considere-se uma escala de avaliações, numa escola, de 0 a 20 (de zero a vinte) valores. Considere-se a negativa uma avaliação abaixo de 10 e a positiva acima deste valor, inclusive. Utilizar-se-á apenas e tão-somente, neste exemplo, avaliações inteiras (números inteiros).

```

program avaliacoas;
uses crt;
var nota : integer;

begin
    write('Escreva a avaliacao, inteira, de 0 a 20: ');
    readln(nota);
    case nota of
        // falta a selecção de casos
    else writeln('Nota invalida');
    end;
    readln;
end.

```

Para seleccionar os casos, pode-se utilizar a noção de **intervalo***, existente no Pascal. Evita-se escrever o caso de ser 0, de ser 1, 2, 3 e por aí adiante. Se se quiser que o caso de a avaliação ser "FRACO" seja no intervalo de 0 a 3, inclusive, pode-se escrever este intervalo como **0..3**

* - Mais propriamente Conjunto, só abordado na Parte V.

Então, faça-se a selecção de casos no programa:

```

program avaliacoas;
uses crt;
var nota : integer;

begin
    write('Escreva a avaliacao, inteira, de 0 a 20: ');
    readln(nota);
    case nota of
        0..3 : writeln('Fraco');
        4..9 : writeln('Insuficiente');
        10..12 : writeln('Pouco Suficiente');
        13..15 : writeln('Suficiente');
        16..18 : writeln('Bom');
        19..20 : writeln('Excelente')
    else writeln('Nota invalida');
    end;
    readln;
end.

```

4. Avaliações 2 – uso do Repeat... Until...

Pegue-se no programa anterior e acrescentasse-lhe dois ciclos **Repeat... Until...** Um para analisar a nota assim que é introduzida, e outro para controlar o fecho do programa: caso o utilizador escreva "1" o programa volta a pedir uma avaliação, caso contrário fecha o programa.

A estrutura para analisar a introdução da avaliação terá de ser a seguinte:

```
repeat
  write('Escreva a avaliacao, inteira, de 0 a 20: ');
  readln(nota);
until (nota>=0) and (nota<=20);
```

Pode-se ainda avisar o utilizador caso falhe a introdução da avaliação com uma estrutura de decisão:

```
repeat
  write('Escreva a avaliacao, inteira, de 0 a 20: ');
  readln(nota);
  if not ((nota>=0) and (nota<=20)) then begin
    writeln('Erro! Nao esta no intervalo 0 a 20!');
    writeln;
  end;
until (nota>=0) and (nota<=20);
```

Para controlar o fecho do programa, necessita-se de uma variável de controlo, à qual chamar-se-á fecho e será do tipo **integer**.

```
repeat
  // programa todo
  write('"1" para reiniciar, outro número para sair: ');
  readln(fecho);
until (fecho<>1);
```

Implemente-se agora estas duas estruturas no programa:

```
program avaliacoes;
uses crt;
var nota, fecho : integer;

begin
  repeat
    repeat
      write('Escreva a avaliacao, inteira, de 0 a 20: ');
      readln(nota);
      if not ((nota>=0) and (nota<=20)) then begin
        writeln('Erro! Nao esta no intervalo 0 a 20!');
        writeln;
      end;
    until (nota>=0) and (nota<=20);
    case nota of
      0..3 : writeln('Fraco');
      4..9 : writeln('Insuficiente');
      10..12 : writeln('Pouco Suficiente');
      13..15 : writeln('Suficiente');
      16..18 : writeln('Bom');
      19..20 : writeln('Excelente');
    else writeln('Nota invalida');
    end;
    writeln;
    write('"1" para reiniciar, outro número para sair: ');
    readln(fecho);
  until (fecho<>1);
end.
```

Propostas de Exercícios

1. Crie a sua própria calculadora. Contudo, o utilizador terá de escolher a operação a realizar e, no caso de esta ser a divisão, o programa terá de impedir o utilizador de introduzir denominador 0 (zero), obrigando-o a introduzir outro valor para tornar a divisão possível. Mostre uma mensagem de erro neste caso.

As variáveis para os valores das operações a realizar terão de ser reais, e a que selecciona a operação deverá ser uma string.

Mostre ao utilizador o maior número de informações possíveis sobre a calculadora. Mas não sobre o que o programa está a processar no momento, como "A realizar a soma".

Normas:

Limite máximo de linhas de código (exclusive comentários e linhas em branco): 40 linhas.

5. Ordenação crescente de uma lista – uso do ciclo For... To... Do...

Para ordenar uma lista de valores, peça-se ao utilizador para criar uma lista de dez números inteiros, e de seguida mostra-se a lista por ordem crescente.

Comece-se por definir as variáveis necessárias:

```
program crescente;
var lista : array [1..10] of integer; // lista de 10 inteiros
    i, j : integer; // dois contadores para os ciclos FOR

begin

end.
```

O programa será feito, doravante, por partes. Começa-se por pedir os 10 valores da lista ao utilizador:

```
writeln('INTRODUZA OS 10 VALORES DA LISTA:');
for i:=1 to 10 do begin
    write('Valor nº', i, ': ');
    readln(lista[i]);
end;
```

Para ordenar a lista pode-se utilizar o seguinte método.

Começando pelo primeiro valor, analisa-se cada número com todos os restantes valores da lista. Caso encontre um menor que este, trocam-se directamente. De seguida, analisa o segundo valor, e compara-o com os seguintes, mas **NÃO** com os anteriores (neste caso, o primeiro). O processo repete-se até ao último valor da lista.

Exemplo:

- Lista: 7 » 9 » 5 » 2

Processo:

1. "Pega" no primeiro valor da lista e compara:

- 9 < 7 – falso: 7 » 9 » 5 » 2
- 5 < 7 – verdade: 5 » 9 » 7 » 2

Agora o primeiro valor é o 5, e a análise continua:

- 2 < 5 – verdade: 2 » 9 » 7 » 5

Como se verifica, o número mais baixo da lista ficou em primeiro lugar.

2. Compara com o segundo valor da lista:

- 7 < 9 – verdade: 2 » 7 » 9 » 5

Agora o segundo valor é o 7.

- 5 < 7 – verdade: 2 » 5 » 9 » 7

3. Agora, com o terceiro valor:

- 7 < 9 – verdade: 2 » 5 » 7 » 9

E termina aqui a análise. Como se verifica, este processo coloca a lista toda por ordem crescente.

Em código, necessitar-se-á de uma variável auxiliar para realizar a troca, chamada aux:

```
for i:=1 to 10 do begin
  for j:=i to 10 do begin
    if (lista[j]<lista[i]) then begin
      aux:=lista[j];
      lista[j]:=lista[i];
      lista[i]:=aux;
    end;
  end;
end;
```

Só resta mostrar a lista por ordem crescente obtida:

```
write('Lista por ordem crescente: ');
for i:=1 to 10 do begin
  write(lista[i],'; ');
end;
```

Implementando estas partes no programa final:

```
program crescente;
var lista : array [1..10] of integer; // lista de 10 inteiros
    i, j : integer; // dois contadores para os ciclos FOR
    aux : integer; // auxiliar para troca de valores em lista

begin
  writeln('INTRODUZA OS 10 VALORES DA LISTA:');
  for i:=1 to 10 do begin // lê a lista
    write('Valor nº', i, ': ');
    readln(lista[i]);
  end;

  for i:=1 to 10 do begin // coloca a lista por ordem crescente
    for j:=i to 10 do begin
      if (lista[j]<lista[i]) then begin
        aux:=lista[j];
        lista[j]:=lista[i];
        lista[i]:=aux;
      end;
    end;
  end;

  write('Lista por ordem crescente: ');
  for i:=1 to 10 do begin // mostra a lista, agora ordenada
    write(lista[i],'; ');
  end;
  writeln;
  write('ENTER para sair. . .');
  readln;
end.
```

Propostas de Exercícios

2. Crie um contador de avaliações positivas de uma turma.

O utilizador deverá introduzir o número de alunos que a turma tem, e de seguida o programa pedirá a avaliação de cada aluno. Para tal, o programa deverá ter um controlo da avaliação introduzida, para que esta esteja dentro de intervalo 0 a 20 valores. No final, o programa irá devolver quantas avaliações positivas existem, apresentando a percentagem, arredondada às décimas.

Normas:

O programa não deverá ter mais do que um ciclo `for`;

Limite máximo de linhas de código (exclusive comentários e linhas em branco): 20 linhas.

3. Crie um programa que debite o maior de todos os n valores introduzidos pelo utilizador, sendo os n números a introduzir definidos pelo utilizador. O limite mínimo é 5 e o máximo é 50.

Normas:

O programa não deverá ter nenhum ciclo `for`;

Limite máximo de linhas de código (exclusive comentários e linhas em branco): 25 linhas.

6. Contagem de espaços e palavras numa frase – uso de string

Imagine-se que é necessário saber quantos espaços e quantas palavras tem uma frase: é necessário um programa que conte o número de espaços que se encontra numa frase.

Pede-se ao utilizador para escrever uma frase, e o programa dirá o número de espaços e de palavras que encontrou.

Primeiro, o contador de espaços:

```
program contador;
uses crt;
var frase : string;
    i, contaespacos, contapalavras : integer;

begin
    write('Introduza uma frase a sua escolha: ');
    readln(frase);
    for i:=1 to length(frase) do begin // contador de espaços
        if (frase[i]=' ') then contaespacos:=contaespacos+1;
    end;
end.
```

Uma palavra está entre espaços, e esta é a base para a análise de palavras neste programa:

```
program contador;
uses crt;
var frase : string;
    i, contaespacos, contapalavras : integer;

begin
    write('Introduza uma frase a sua escolha: ');
    readln(frase);

    contaespacos:=0;
    for i:=1 to length(frase) do begin // contador de espaços
        if (frase[i]=' ') then contaespacos:=contaespacos+1;
    end;

    contapalavras:=0;
    if (frase[1]=' ') and (frase[2]<>' ') then contapalavras:=1;
    // Poderá haver um espaço a começar a string "frase". Este código prevê esta situação.
    if frase[1]<>' ' then contapalavras:=1;
    // verifica igualmente se o primeiro caracter NÃO é um espaço,
    //contando então uma palavra.
```

```

for i:=2 to length(frase) do begin
    // análise da restante string, já com a existência da primeira palavra prevista
    if (frase[i]=' ') and (upcase(frase[i+1]) in ['A'..'Z']) then
        contapalavras:=contapalavras+1;
        // se o actual character da frase é um espaço e o próximo é uma letra,
        // então acabou a palavra - soma 1 ao contador}
end;

writeln('A frase contém ',contapalavras,' palavras, assim como ',contaespacos,
                                             ' espaços');

readln;
end.

```

Propostas de Exercícios

4. Crie um programa que elimine os espaços a mais.

O utilizador introduz uma frase à sua escolha, e o programa irá eliminar os espaços a mais, reduzindo-os a um só, entre cada palavra.

Normas:

Terá de utilizar somente ciclos `for`;

Limite máximo de linhas de código (exclusive comentários e linhas em branco): 30 linhas.

7. Cálculo da tangente – uso de uma função

Para treinar a criação de uma função, um bom exemplo é o da tangente de um número (ângulo), já que o Pascal não tem uma.

Ter-se-á de ter em atenção que $\cos(a)$ não pode ser 0 (zero), pois:

$$\text{tg}(a) = \frac{\text{sen}(a)}{\cos(a)}$$

Nesse caso, o programa dirá que a tangente não é possível. Para saber se é possível, criar-se-á outra função que indica tal.

```

program tangente;
uses crt;
var a : real;

function tan(angulo:real):real; // tangent = seno / co-seno
begin
    tan:=sin(angulo)/cos(angulo);
end;

function tangente_possivel(angulo:real):boolean;
// a tangente é possível se o co-seno for diferente de zero
begin
    if (cos(angulo)=0) then tangente_possivel:=false
    else tangente_possivel:=true;
end;

begin
    write('Introduza angulo, em radianos: ');
    readln(a);
    if tangente_possivel(a) then writeln('Tangente de ',a:0:10,' : ',tan(a):0:10)
    else writeln('Tangente de ',a:0:10,' : infinito');
    readln;
end.

```

Propostas de Exercícios

5. Crie um programa que calcule o factorial de um número. O programa deverá analisar se o valor introduzido é inteiro ou não, e se é positivo, ou seja, a variável que lê o número é real. Deverá prever a situação especial do factorial de zero.

Normas:

Terá de utilizar uma função para cálculo do factorial;

Limite máximo de linhas de código (exclusive comentários e linhas em branco): 25 linhas.

PARTE III

Funções e Procedimentos padrão. Conversão. Personalização gráfica.

Após uma introdução básica e uma parte essencialmente prática e de desenvolvimento da leitura lógica de um programa em Pascal, é altura de continuar com uma nova parte novamente teórica.

Os **Desafios** doravante apresentados pretendem ser exercícios mais complexos que levam o “aprendiz” a pensar mais para poder chegar à conclusão.

Nesta parte do Tutorial, dar-se-á ênfase à utilização de operadores para realização de cálculos (revisão prática), recorrendo sempre que possível a procedimentos e funções para que estes sejam treinados e seja entendida a sua importância na estruturação de um bom programa. Esta parte também começará a dar ênfase às boas práticas no acto da programação para a fácil leitura e interpretação de um programa Pascal.

Começar-se-á igualmente com **optimização** de nível básico, e com comandos de coloração e outros direccionados a strings.

1. Operações numéricas gerais

Para além das quatro operações básicas, existem outras suportadas pelo Pascal. Muitas foram já abordadas previamente, mas seguem-se como breve revisão prática.

SIN, COS e ARCTAN

```
program operacoes_01;
uses crt;
var a : real;

begin
  write('Introduza angulo em RADIANS: ');
  readln(a);
  write('Seno do angulo: ', SIN(a):0:3); // valor arredondado a 3 casas decimais
  write('Co-seno do angulo: ', COS(a):0:3);
  write('Arco-tangente: ', ARCTAN(a):0:3);
  readln;
end.
```

ROUND, TRUNC e ABS

- **round** – arredonda o número às unidades – útil para converter um Real em Integer;
- **trunc** – obtém o valor inteiro no número, sem o arredondar – valor **truncado**;
- **abs** – devolve o valor absoluto do número.

Exemplo da sua utilização:

```
program operacoes_02;
var r : real;

begin
  write('Introduza numero real: ');
  readln(r);
  write('Arredondado às unidades: ', ROUND(r));
  // equivale a "r:0:0". Contudo, poderá gravar o valor arredondado numa variável,
  // e "r:0:0" não o faz - tal não é permitido.
  write('Parte inteira: ', TRUNC(r));
  write('Parte fraccionária: ', ABS(r-TRUNC(r)));
  // considera-se a parte fraccionária sempre POSITIVA, daí a utilização da função ABS.
```

```
write('Valor absoluto: ',ABS(r));
write('Absoluto da parte inteira: ',ABS(TRUNC(r)));
readln;
end.
```

DIV e MOD

Estas são operações muito úteis, relacionadas com a divisão. Relembrando a Parte I do tutorial:

- **DIV** – devolve o resultado de uma divisão, em número inteiro, truncado: só permite a utilização de variáveis Integer;
- **MOD** – devolve o resto de uma divisão: só permite a utilização de variáveis Integer;

```
program operacoes_03;
uses crt;
var r, s : real;

begin
  write('Introduza dois valores reais: ');
  readln(r, s);
  write('Divisao "normal": ',r/s);
  // não é prevista a situação de s=0, para simplificação da leitura do programa.
  write('Div. Inteira: ',ROUND(r) DIV ROUND(s));
  write('Resto: ',ROUND(r) MOD ROUND(s));
  readln;
end.
```

2. Optimização básica

Optimização é o processo que permite reduzir o número de linhas de código: o programa realiza exactamente as mesmas operações com menos código, ou seja, o executável terá um tamanho menor, e poderá ser, porventura, mais rápido de ser processado.

As primeiras optimizações que irão ser feitas serão na estrutura de decisão **If... Then... Else...** A regra desta estrutura, como já foi visto, é a seguinte:

```
if (condição) then begin
  comandosA;
end
else begin
  comandosB;
end;
```

No caso de comandosA ou comandosB serem **uma e uma só** instrução, optimiza-se a estrutura das seguintes formas:

```
// comandosA é uma só instrução:
if (condição) then comandoA
else begin
  comandosB;
end;

// comandosB é uma só instrução:
if (condição) then begin
  comandosA;
end
else comandoB;

// comandosA e comandosB são uma só instrução:
if (condição) then comandoA
else comandoB;
```

Por exemplo:

```
readln(numero);
if (numero<0) then write('Numero negativo.')
else begin
    if (numero>0) then write('Numero positivo')
    else write('Numero nulo');
end;
```

Agora, o mesmo se aplica a qualquer ciclo de repetição. No caso de o código a executar forem tão-somente uma instrução, ficará assim:

```
// ciclo WHILE
while (condição) do comandoA;

// Ciclo FOR
for i:=1 to 10 do comandoB;
for i:=1 downto 10 do comandoC;
```

Ou seja, em todos os casos, begin e end são as palavras reservadas omitidas.

A única estrutura que não tem optimização possível é o ciclo **Repeat... Until...**

3. Facilitação da leitura de um programa

Imagine-se o seguinte caso. Pretende-se que o utilizador de um programa introduza um número real, em frente à mensagem que lhe colocarmos. Por exemplo, algo como:

```
write('Introduza um numero real: ');
readln(numero);
```

Como o cursor fica à frente da mensagem e o número será introduzido em frente deste, o próprio código poderá dar a entender este facto da seguinte forma:

```
write('Introduza um numero real: '); readln(numero);
// o cursor fica à frente da mensagem
```

Ou seja, o código em si dá a entender que o programa esperará o valor real (variável numero) à frente da mensagem. Isto é possível pois o ponto e vírgula (;) é o símbolo que faz a separação dos códigos.

Por esta razão é que, no código que antecede um **else** *nunca* é precedido de ";", pois o programa só assim fica a saber que a condição **if** ainda não acabou e que encontrará imediatamente a seguir um *senão* (**else**).

A própria optimização estudada anteriormente facilita em muito a leitura de um programa, pois há menos "poluição visual" com uma série de begins e ends desnecessários. A leitura torna-se mais clara.

O uso de procedimentos e funções facilitará igualmente a leitura do código, para além de o optimizar. As seguintes duas linhas de código fazem exactamente o mesmo, contudo uma utiliza funções e outra não:

NB: As funções aqui apresentadas estão programadas na Parte II do tutorial.

```
// sem recurso a funções:
if (COS(a)<>0) then writeln('A tangente e: ',SIN(a)/COS(a):0:3)
else writeln('Tangente impossível');

// com recurso a funções:
if tangente_possivel(a) then writeln('A tangente e: ',tan(a):0:3);
else writeln('Tangente impossível');
```

Com recurso a funções, consegue-se ler directamente o seguinte:

Se a tangente de a é possível então calcula-a, se não diz que é impossível.

Compare-se com o caso em que não há recurso a funções. A leitura é dificultada.

Este é meramente um caso simplista. Casos mais complexos revelarão melhor a importância das funções e dos procedimentos na leitura fácil de um programa, mas tais casos não estão no âmbito deste Tutorial devido à sua complexidade (número de linhas de código envolvidas – o foco desta parte seria desviada para o entendimento do programa em si).

4. A fórmula resolvente – programa completo – uso de funções

Pretende-se um programa que realize a Fórmula Resolvente, recorrendo a Procedimentos e Funções variados.

Tópicos para o programa:

- Função para o cálculo do Binómio Discriminante;
- Função para o cálculo de ambos os X (soluções da equação quadrática);

Cumprindo os variados tópicos, deve-se primeiramente definir como funcionará cada Função e cada Procedimento.

```

program formula_resolvente;
var a, b, c : real;
    i : integer;

function resolucao_possivel(d1:real) : boolean; // verifica se há solução real
begin

end;

function d(a1:real; b1:real; c1:real) : real; // calcula o binómio discriminante
begin

end;

function x(i:integer; a1:real; b1:real; c1:real) : real; // calcula o X nº "i".
begin

end;

begin
    writeln('Ax^2 + Bx + C = 0');
    writeln('Introduza os tres coeficientes por ordem: ');
    readln(a, b, c);
end.

```

Agora que o esqueleto do programa está montado, resta programá-lo de acordo com as funções de que se dispõe.

Para treinar a leitura lógica de um programa que utilize funções, a explicação não será alongada. Claro que o exemplo seguinte tem funções utilizadas com um certo abuso, mas tal foi propositado.

```

program formula_resolvente;
var a, b, c : real;
    i : integer;

function resolucao_possivel(d1:real) : boolean; // verifica se há solução real
begin
    if (d1>=0) then resolucao_possivel := true // se o Bin. Disc. For maior ou igual que
    zero, há soluções reais
    else resolucao_possivel := false;
end;

function d(a1:real; b1:real; c1:real) : real; // calcula o binómio discriminante
begin
    d:=SQR(b1) - 4*a1*c1;
end;

```

```
function x(j:integer; a1:real; b1:real; c1:real) : real; // calcula o X n° "j".
begin
  case j of
    1 : x := (-b1 + SQRT( d(a1, b1, c1) )) / (2*a1);
    2 : x := (-b1 - SQRT( d(a1, b1, c1) )) / (2*a1);
  end;
end;

begin
  writeln('Ax^2 + Bx + C = 0');
  writeln('Introduza os tres coeficientes por ordem: ');
  readln(a, b, c);
  if resolucao_possivel( d(a, b, c) ) then begin
    // se a resolução é possível (sabendo-se tal pelo Discriminante), então...
    for i:=1 to 2 do write('Solucao ',i,': ', x(i, a, b, c):0:3 );
  end else begin
    write('Impossivel em R.');
```

Desafio 1

Crie um programa que leia o nome do utilizador e apresenta-o com os caracteres por ordem inversa. Por exemplo:

"Thoga 31" passará a ser "13 agoht"

Contudo, terá de criar uma **função** para 1) ler uma variável string, denominada ler e 2) escrever um texto, denominada escrever;

Normas: livre

5. Conversão de variáveis

Existem, claro está, funções que permitem converter as variáveis de uns tipos para outros.

É possível converter de Real para String, de String para Real, de Integer para Real e de Real para Integer. Existem outras variáveis, contudo, que não são abordadas de momento.

5.1. Real/Integer – String

Sejam dadas as variáveis r, Real, e s, String. Sejam igualmente as variáveis erro e i, Integer.

```
str(s, r, erro);
// Converter a string "s" para real, "r".
// Se tal não for possível, indicar a posição do erro, em "erro".
```

A **posição do erro** consiste no índice da string no qual existe o erro. Exemplo:

- 574P5 – o erro está na posição 4: "P".

```
str(s, i, erro);
// Converter a string "s" para integer, "i".
// Se tal não for possível, indicar a posição do erro, em "erro".
```

A **posição do erro** consiste no índice da string no qual existe o erro. Exemplo:

- 5.915 – o erro está na posição 2: "."; um Integer não tem parte decimal.

```
val(r, s);
// Converter o real "r" para string, "s".
val(i, s);
// Converter o integer "i" para string, "s".
```

A conversão de `real/integer` para `string` não implicará erro nenhum, caso o processo seja correctamente efectuado na CPU, o que não é previsto o contrário.

5.2. Real – Integer

Considerando as mesmas variáveis:

```
r := i;
```

Não são necessárias normas para esta conversão, visto que um número inteiro (Integer) não passa de um caso especial do universo dos reais (Real). Logo são intimamente compatíveis nesta, e só nesta, conversão.

Já o inverso não se verifica, pois um número real pode não pertencer ao universo dos inteiros. Para esta conversão, é necessário arredondar o Real, e o Pascal só reconhece esta operação com um de dois operadores: `ROUND` e `TRUNC`, cujas diferenças já foram previamente explicadas.

```
i := ROUND(r);  
i := TRUNC(r);
```

6. Coloração de texto e fundo

O sistema de cores que o Pascal suporta é o seguinte:

- Para cor do **texto**: 4 bits (16 cores);
- Para a cor do **fundo**: 3 bits (8 cores);

Os códigos que permitem a aplicação de cores são os seguintes:

- Para cor do **texto**: `textcolor`;
- Para a cor do **fundo**: `textbackground`;

A cor predefinida do texto é a nº 7, e a de fundo é a nº 8.

```
textcolor(7);  
textbackground(8);
```

Para conhecer as cores do Pascal, pode-se construir o seguinte programa:

```
program cores;  
uses crt;  
var i : integer;  
  
begin  
  writeln('Cores de TEXTO:');  
  for i:=1 to 16 do begin  
    textcolor(i);  
    write(' Cor ',i,' ');  
  end;  
  writeln;  
  writeln('Cores do FUNDO:');  
  textcolor(15); // branco  
  for i:=1 to 8 do begin  
    textbackground(i);  
    write(' Cor ',i,' ');  
  end;  
  writeln;  
end.
```

Desafio 2

Crie um programa que construa uma tabela em que, em cada coluna, fica uma cor de fundo, e em cada linha uma cor de texto, para que possam ser analisadas todas as combinações fundo/texto possíveis.

Normas: livre

7. Caracteres ASCII

O sistema de caracteres do Pascal é o ASCII. Dependendo de cada país e/ou sistema, a tabela pode sofrer algumas modificações, as quais o Pascal é sensível pois ele recorre ao sistema ASCII do próprio computador e não a uma tabela ASCII própria.

Para se conhecer a tabela ASCII de cada programa, é possível criar um programa que a debite. Contudo, é apresentada uma tabela padrão na Parte VI.

Para saber o carácter de ordem *i* da tabela ASCII:

```
write( CHAR(i) );
```

Já para saber a ordem de um carácter:

```
write( ORD(caracter) );
```

Sendo *caracter* uma variável do tipo *char*.

No caso de *string*, para se saber a ordem de **cada** carácter dever-se-á realizar o seguinte ciclo:

```
for i:=1 to length(s) do write(ORD(s[i]), ' ');
```

(*i* – variável Integer)

Só com o método *CHAR* é possível escrever acentuações em determinados IDEs/compiladores.

Desafio 3

Crie um programa que construa uma tabela ASCII, indicando a *ordem* seguida do *caracter* correspondente.

Normas: livre

Desafio 4

Crie, num programa, tão-somente duas funções que façam uma conversão especial.

Muitas vezes, o valor booleano *True* é associado ao número 1, e *False* ao número 0. Então, uma das funções converte um booleano no seu correspondente numérico. A outra função realizará o inverso.

Normas: livre

Propostas de Exercícios

6. Crie uma calculadora que realize as quatro operações básicas, com previsão de erros, como o da divisão. Estes deverão ser escritos a vermelho vivo. Os textos deverão ser a branco e os resultados a amarelo vivo.

Faça um programa em que, no fim, se possa optar se se deseja realizar novo cálculo ou se se deseja sair.

Normas:

Limite de linhas de código (exclusive comentários e linhas em branco): 70 linhas.

7. Um programa deverá desenhar a bandeira da França. Utilize ciclos `for`.

Faça um procedimento para desenhar a bandeira, onde as variáveis de entrada serão a cor a utilizar, o número de colunas e o número de linhas a desenhar.

Normas:

Limite de linhas de código (exclusive comentários e linhas em branco): 30 linhas.

PARTE IV

Registos. Ficheiros de Texto Simples.

Segue-se agora para uma parte do Pascal mais complexa e que começa a mostrar novas potencialidades desta linguagem de programação. Trabalhar-se-á com variáveis do tipo Record, extremamente importante em Pascal, e dar-se-ão as bases sobre as variáveis Text, variáveis que permitem criar ficheiros de texto simples. Serão dados de igual forma códigos úteis sobre strings e sobre o teclado e o cursor.

1. A variável Record

Estas variáveis são extremamente úteis para, por exemplo, programas de registos. Uma variável Record tem várias “subvariáveis”, designadas por **campos**. Eis um exemplo da declaração de uma variável Record:

```
var registo : record
    nome : string;
    idade : integer;
    sexo : char;
end;
```

Para pedir ao utilizador, por exemplo, para registar a idade, far-se-á do seguinte modo:

```
writeln('Registe a idade: '); readln(registo.idade);
```

Ou seja, a variável Record faz-nos lembrar um formulário onde devemos preencher os vários *campos*, por exemplo num CV.

A variável Record é um **formulário**, e as suas várias “subvariáveis” (campos) são os **campos** desse formulário.

É possível criar uma “resma” de formulários sob a forma de um array de Record – isto revela que é possível criar milhares de campos em memória, organizados por grupos individuais, de uma forma muito simples e eficaz, sem recorrer a milhares de variáveis. Ou seja, em vez de termos as variáveis nome1, nome2, nome3, idade1, idade2, idade3, sexo1, sexo2 e sexo3, temos três registos (Record), cada um com os seus campos idade, nome e sexo. Em código:

```
type tipo_reg = record
    nome : string;
    idade : integer;
    sexo : char;
end;

var registo : array [1..3] of tipo_reg;
```

Então, pode-se preencher os 3 formulários (array [1..3]) recorrendo a um ciclo for.

```
for i:=1 to 3 do begin
    writeln('A FAZER REGISTO NUMERO ',i);
    write('Nome: '); readln(registo[i].nome);
    repeat
        write('Sexo (M/F): '); readln(registo[i].sexo);
        //só aceita os caracteres "M" e "F":
    until (upcase(registo[i].sexo) = 'M') or (upcase(registo[i].sexo) = 'F');
    repeat
        write('Idade: '); readln(registo[i].idade);
        // a idade é positiva ou nula:
    until (registo[i].idade >= 0);
    writeln('REGISTO ',i,' DE 3 EFECTUADO.');
```

```
writeln;  
end;
```

Da mesma forma, estes registos podem ser escritos com um ciclo for:

```
for i:=1 to 3 do begin  
  writeln('A VER REGISTO NUMERO ',i);  
  writeln('Nome: ',upcase(registo[i].nome));  
  writeln('Sexo: ',upcase(registo[i].sexo));  
  writeln('Idade: ',registo[i].idade);  
  writeln;  
end;
```

Assim acabou por ser criado um programa completo que realiza um registo básico de 3 pessoas, escrevendo o que foi introduzido no final. Este fica, então, como um programa exemplo sobre este tipo de variáveis, essenciais em Pascal:

```
program variavel_record;  
uses crt;  
  
type tipo_reg = record  
  nome : string;  
  idade : integer;  
  sexo : char;  
end;  
  
var registo : array [1..3] of tipo_reg;  
    i : integer;  
  
begin  
  for i:=1 to 3 do begin // REGISTO  
    writeln('A FAZER REGISTO NUMERO ',i);  
  
    write('Nome: '); readln(registo[i].nome); // registo do NOME  
  
    repeat // registo controlado do SEXO  
      write('Sexo (M/F): '); readln(registo[i].sexo);  
    until (upcase(registo[i].sexo) = 'M') or (upcase(registo[i].sexo) = 'F');  
  
    repeat // registo controlado da IDADE  
      write('Idade: '); readln(registo[i].idade);  
    until (registo[i].idade >= 0);  
  
    writeln('REGISTO ',i,' DE 3 EFECTUADO.');
```

```
    writeln;  
  end;  
  
  writeln;  
  writeln;  
  
  for i:=1 to 3 do begin // ESCRITA  
    writeln('A VER REGISTO NUMERO ',i);  
    writeln('Nome: ',upcase(registo[i].nome));  
    writeln('Sexo: ',upcase(registo[i].sexo));  
    writeln('Idade: ',registo[i].idade);  
    writeln;  
  end;  
  
  readln; // pausa até ENTER  
end.
```

Propostas de Exercícios

8. Crie um programa que registe os dados sobre uma turma inteira e faça a estatística básica. Serão pedidos o número de alunos, e o nome e as avaliações a Matemática e a Português de cada um. No final, o programa faz a estatística e diz quais os alunos aptos a ir a Exame Nacional (média igual ou superior a 9,5) de cada disciplina, qual a média da turma a Matemática e a média a Português.

Normas:

Deverá ser utilizado um só `Record`, que inclua todos os campos;
Recorra a ciclos `for` para percorrer os campos;
No máximo, só poderão existir 30 alunos;
Limite de linhas de código (exclusive comentários e linhas em branco): 100 linhas.

1.1. A cláusula `with`

Existe um código, também denominado, neste caso, de **cláusula**, que permite “pegar” num `Record` e trabalhar com ele, sem que se tenha de escrever constantemente o nome do registo. Por exemplo:

```
with registo[2] do begin
    nome := 'Thoga31';
    idade := 0;
    sexo := 'M';
end;
```

Substitui:

```
registo[2].nome := 'Thoga31';
registo[2].idade := 0;
registo[2].sexo := 'M';
```

Ou seja, simplifica muito a escrita de código, e permite maior legibilidade.

2. Códigos úteis sobre o teclado e o cursor

2.1. Posição do cursor

Até ao momento, os programas têm sido direccionados para as consolas DOS. Como se sabe, a consola DOS tem sempre o **cursor** a piscar, numa determinada **posição**. Em Pascal é possível controlar esta posição através de um comando extremamente simples.

Considere-se a janela como um gráfico cartesiano, só de valores inteiros. Seja X a abcissa (horizontal) e Y a ordenada (vertical). A posição inicial do cursor na consola é no canto superior esquerdo: X=1 e Y=1. Ande-se duas linhas para baixo: X=1 e Y=3. Ande-se com o cursor nove colunas para a frente (direita): X=10 e Y=3.

Se quisermos posicionar o cursor nesta última posição:

```
gotoxy(10, 3);
```

CÓDIGO: posicionamento do cursor na consola DOS

gotoxy(x, y)

Este código só é utilizável quando declarada a biblioteca `crt`. Caso contrário, é um *código não declarado*.

2.2. Determinação da tecla premida

É possível gravar a qualquer altura qual a tecla premida por um utilizador. Esta função é extremamente útil na criação de menus apelativos. O utilizador em vez de escrever, por exemplo, "1" e, de seguida, premir ENTER, basta-lhe-á premir a tecla "1" e, se for uma opção válida do menu apresentado, avança imediatamente, sem ser necessário premir ENTER. Por exemplo:

```
uses crt; // biblioteca OBRIGATÓRIA
var tecla : char;

// (. . .)

writeln('MENU');
writeln('1 > Repetir operacao');
writeln('2 > Sair');

repeat
    tecla := readkey;
    // grava na variável TECLA qual a tecla (key) premida pelo utilizador
until (char(tecla) = '1') or (char(tecla) = '2');
```

O código **readkey** determina a tecla premida pelo utilizador, gravando-a numa variável **char**.
Este código só é utilizável quando declarada a biblioteca **crt**. Caso contrário, é um *código não declarado*.

Propostas de Exercícios

- Crie uma calculadora que grave num Record, em campos distintos, os dois termos da operação, e o operador. O utilizador deverá escolher o operador sem ter de premir ENTER, segundo uma lista dos operadores disponíveis (soma, subtracção, multiplicação e divisão). Preveja situações de erro, e permita que o utilizador as corrija. O utilizador, com um novo menu, sem recurso ao ENTER, irá escolher se deseja sair do programa ou realizar novo cálculo. Utilize coloração de texto variada, e não de fundo. O ecrã deverá ter fundo cinza claro.

Normas:

Deverá ser utilizado um só Record;

Limite de linhas de código (exclusive comentários e linhas em branco): 100 linhas.

3. A variável Text – escrita e leitura de ficheiros de texto simples – parte 1

O Pascal tem a capacidade de trabalhar com ficheiros. Existem duas variáveis para o fazer, contudo vamos abordar apenas aquela que permite escrever e ler ficheiros de texto simples, ou seja, ficheiros cujo conteúdo não passa de texto simples, não formatado: é a variável **Text**, cujo nome denuncia o tipo de ficheiros que vai trabalhar.

O funcionamento desta variável é em tudo diferente das variáveis com que já trabalhamos até agora.

Vamos, então, estudar esta variável por partes e com exemplos práticos.

O primeiro programa apresentado cria um ficheiro **TXT** e grava nele o texto "Hello World! For Pascal TEXT!". O ficheiro é nomeado de **texto.txt**.

```
program escrever_ficheiro;
var ficheiro : text;

begin
    assign(ficheiro, 'texto.txt');
    // "assina" em memória o ficheiro de texto simples, e cria-o junto do executável
    rewrite(ficheiro);
    // ordena a re-escrita do ficheiro - caso o ficheiro já exista e contenha texto,
    // este será eliminado
    write(ficheiro, 'Hello World! For Pascal TEXT!');
    // escreve no ficheiro o texto
```

```
close(ficheiro);
// "encerra" o ficheiro em memória
end.
```

Quando se tenta aceder a um ficheiro de texto simples que esteja "assinado" em memória por um programa deste género, aparecerá a famosa Message Box a dizer que o ficheiro está a ser utilizado por outro processo. Isto ajuda a explicar o que acontece no sistema aquando o código `assign`.

Agora, vamos criar um novo programa, que ficará no mesmo directório, e que vai abrir o ficheiro anterior e contar o número de espaços do texto que contém:

```
program ler_ficheiro;
var ficheiro : text;
    s : string;
    contador, i : integer;

begin
    assign(ficheiro, 'texto.txt');
    reset(ficheiro);
    // "reinicia" o ficheiro, ou seja, acede-lhe mas não pode re-escrever dados nele,
    // só pode ler
    readln(ficheiro, s);
    contador := 0;
    for i:=1 to length(s) do if s[i] = ' ' then contador += 1;
    // a linha de código anterior recorre à Optimização
    writeln('Existem ', contador, ' espacos no ficheiro.');
```

```
close(ficheiro);
readln;
end.
```

Desafio 5

Investigue sobre o CV (*Curriculum Vitae*) europeu – o **Europass**.

Crie um programa que crie um CV deste modelo, cumprindo todos os campos, excepto o da Fotografia.

O programa irá registar os dados do utilizador e, de seguida, irá mostrar por passos o que o utilizador introduziu. No caso de confirmação, deverá gravar o CV num ficheiro `TXT` cujo nome será "CV_nome", onde "nome" é o nome do utilizador, sem espaços. Os campos não preenchidos não deverão aparecer no ficheiro.

Caso o utilizador pretenda alterar um campo, tal deverá ser permitido sem que, para tal, seja necessário refazer o CV todo.

Normas:

O CV deverá estar registado num só Record, que inclua Records das partes, e dentro de cada parte os seus campos respectivos;

O CV só será gravado em ficheiro no fim, aquando a confirmação final por parte do utilizador;

O utilizador poderá cancelar o CV a qualquer altura, escrevendo "<cancelar>";

Limite de linhas de código (exclusive comentários e linhas em branco): livre

4. Códigos úteis sobre string

Seja a variável **s** uma `String` com o seguinte texto:

BEM VINDO

```
pos('M', s);
```

Determina a posição do primeiro "M" que encontrar: neste caso, 3.

Devolve um valor `Integer`, a ser gravado numa variável deste tipo.

```
delete(s, 7, 3);
```

Elimina 3 caracteres a partir da posição 7, inclusive.
A string passa a ser "BEM VI".

```
insert('-', s, 4);
```

Insere, na posição 4, o texto "-".
A string passa a ser: "BEM – VINDO".

Propostas de Exercícios

10. Crie um programa em que o utilizador introduza uma string e possa realizar testes variados com ela. A original deverá manter-se intacta, sendo uma string auxiliar a utilizada para as operações.

O utilizador poderá escolher a posição, os caracteres e textos, entre outros, relacionados com os códigos anteriores.

Normas: livre.

5. Aleatoriedade

Uma das capacidades do computador é o cálculo de números aleatórios. Estes números não são propriamente aleatórios - chama-se a este processo **pseudo-aleatoriedade**, isto porque o computador não tem a capacidade de "pensar" num número aleatoriamente. Existem diversos algoritmos, praticamente todos baseados no **tempo**. Este princípio baseia-se, geralmente, num **Timer** do computador em que, quando é requerido um número pseudo-aleatório, este calcula o segundo uma fórmula que inclui os limites mínimo e máximo do conjunto no qual deve estar o número, e o **Timer**. É devolvido então o resultado de um **cálculo**, e não um número totalmente aleatório.

Em Pascal, é necessário, na generalidade dos compiladores, "activar" este **Timer** para que possam ser debitados valores pseudo-aleatórios.

```
randomize;
```

Não existe nenhum procedimento que "pare" o **Timer**. Contudo, esta última instrução pode ser constantemente utilizada sem "efeitos".

O Pascal só admite conjuntos de 0 a **maxint** (i.e., o número máximo inteiro positivo suportado), inclusive.
Para requerer um número pseudo-aleatório que pertença a um conjunto de 0 a *n*, inclusive:

```
random(n);
```

Esta função devolve um número **real**.

Crie-se, então, um pequeno programa que debite em toda a consola um código binário aleatório:

```
program binario;
uses crt;
var i, j : integer;

begin
  randomize;
  for i:=1 to 20 do begin
    for j:=1 to 80 do begin
      gotoxy(j,i);
      write(TRUNC(RANDOM(2)));
    end;
  end;
  readln;
end.
```

É utilizada a função `Trunc` devido ao facto de `Random` devolver números tais como, por exemplo, 0.365, 1.845 (aqui abreviados). Assim, apenas é mostrado o valor inteiro do número, não arredondado - ou 0 ou 1, tal como desejado.

Propostas de Exercícios

11. Simule a Lei dos Grandes Números num programa que irá "lançar" um dado cúbico regular um milhão (1.000.000) de vezes. O programa só deverá gerar como *output* o número de vezes que "saiu" cada face. Tenha em conta que não existe a face 0 (zero), isto por causa do "defeito" do comando `Random` - neste caso, o computador deverá calcular nova face, sem que conte como um lançamento.

Normas: livre.

PARTE V

Conjuntos. Estruturação de um programa em Pascal.

Data e Hora do Sistema.

Nesta parte são abordados os Conjuntos, que são uma estrutura constituível pelo programador, isto é, uma colecção, de elevada potencialidade em Pascal. É abordada mais a fundo uma forma simples de estruturar um programa recorrendo à estrutura *Begin/End*. E, por fim, é dado, por curiosidade, três procedimentos padrão de uma nova biblioteca que permitem determinar a data e a hora do sistema, e obrigar o programa a parar durante um certo tempo antes de avançar.

1. Conjuntos

Parte deste capítulo foi baseado em: GOTTFRIED, Byron S., *Programação em Pascal*, McGraw-Hill, 1994

Algumas frases poderão ser iguais às do livro. Parte dos exemplos é retirado igualmente deste livro.

Isto deve-se ao facto de este livro estar muito bem conseguido no que toca a Conjuntos, tornando-se inigualavelmente claro.

Um **conjunto** é uma **colecção de elementos** simples ordenados, todos do mesmo tipo.

Um conjunto não é fácil de se declarar, mas, anteriormente, já os utilizámos de forma inconsciente, aquando a escrita, por exemplo, de:

```
while not A in [0..9] do
```

[0..9] é um conjunto de reais, que começa em 0 (zero), inclusive, e termina em 9 (nove), inclusive.

Já no programa que analisava as avaliações dos alunos, utilizámos a seguinte notação:

```
case avaliacao of
  0..3 : writeln('Fraco');
  // etc.
end;
```

0..3 representa o intervalo de números reais entre zero e três, inclusive.

1.1. Declaração e utilização

Pode-se criar um conjunto, nomeado (com nome), de uma certa natureza. Teoricamente, há que declarar o conjunto pela seguinte ordem:

- Declarar os *elementos base* como *Type*;
- Declarar o *conjunto* em termos de *elementos base*, como *Type*;
- Declarar o *nome de conjunto*, como *Var*, sendo do tipo *conjunto*.

Parece um processo complicado, e com passos sem sentido. Contudo, esta organização faz parte da estruturação do Pascal como **linguagem estruturada** – pode-se mesmo dizer que “as coisas não aparecem do nada nem por milagre”.

Veja-se a declaração do conjunto em termos teóricos:

```
type elementos_base = (dado1, dado2, ..., dado n);
conjunto = set of elementos_base;
var nome_de_conjunto : conjunto;
```

Agora aplique-se a teoria num exemplo prático:

```
type medidas = (pequeno, medio, grande);
medidas_camisa = set of medidas;
var manga : medidas_camisa;
```

Pode-se definir um conjunto com dados padrão, tal como, por exemplo:

```
type algarismos = set of 0..9;
minusculas = set of 'a'..'z';
```

Contudo, um conjunto pode ser construído. Imagine-se uma calculadora, tal como já foi dada como exercício anteriormente, onde o utilizador pode escolher o operador. Podia-se declarar um conjunto dos operadores, e, no código, simplificar a escrita do programa:

```
program calculadora;
uses crt;

type conjunto_operadores = set of char;

var operadores : conjunto_operadores;
    oper : char;

begin
    oper := ['+', '-', '*', '/'];
    // ...
    repeat
        oper := readkey;
    until char(oper) in operadores;
    // ...
end.
```

Como se constata, `oper` foi declarado como uma **variável**. Contudo, podia ser declarado como uma **constante**, não sendo necessário declarar dois tipos.

```
program calculadora;
uses crt;

const operadores = ['+', '-', '*', '/'];
var oper : char;

begin
    // ...
    repeat
        oper := readkey;
    until char(oper) in operadores;
    // ...
end.
```

O primeiro exemplo é um excelente exemplar da declaração de um conjunto através de tipos, e o segundo exemplo mostra a criação de um conjunto, constante, através de um conjunto individual de elementos ordenados e definidos, inalteráveis.

A utilização deste conjunto simplifica na medida em que, na estrutura Repeat... Until..., torna-se mais legível, ao invés de:

```
repeat
    oper := readkey;
until (char(oper)='+') or (char(oper)='-') or (char(oper)='*') or (char(oper)='/');
```

Os conjuntos são úteis na medida em que é criada uma colecção de elementos ordenados, irrepetíveis, como sendo um **grupo**. Para quem é conhecedor de VB, pode-se fazer a analogia óbvia de que uma **colecção** em Pascal é o mesmo que uma **enumeração** (Enum). Por exemplo, a colecção de operadores criada anteriormente em Pascal seria, em VB, algo como:

```
Private Enum operadores As Char
    "+"
    "-"
    "*"
    "/"
End Enum
```

Seguem-se pequenos exemplos de subconjuntos, aplicando o exemplo das medidas:

```
type medidas = (pequeno, medio, grande);
    medidas_camisa = set of medidas;

var manga_curta, manga_comprida, outra_manga : medidas_camisa;

begin
    // ...
    manga_curta := [pequeno, grande];
    manga_comprida := [pequeno..grande];
    outra_manga := [grande];
    // ...
end.
```

Não é fácil visualizar utilizações “muito” práticas dos conjuntos, em Pascal. Contudo, será construído, mais à frente, um programa para exemplificar a sua utilidade.

Para já, vamos aprender a realizar operações com conjuntos.

1.2. Operações com conjuntos

Ao operar com conjuntos, estamos a falar de três operações básicas, familiares da Teoria de Conjuntos na Matemática:

- Intersecção;
- União;
- Diferença.

Será abordada cada uma delas, por ordem, através de exemplos práticos, todos seguindo a colecção criada anteriormente sobre medidas de camisas.

INTERSECÇÃO

A intersecção baseia-se no princípio de obter os elementos que estão em comum entre dois ou mais conjuntos. Por exemplo, a intersecção entre o conjunto de números {1, 2, 3, 4, 5} e o conjunto {0, 3, 4, 6} é o conjunto {3, 4}. Na matemática, esta operação representa-se por uma **multiplicação**. Assim o é em Pascal:

```
manga_curta := [pequeno, medio] * [medio, grande];
```

Neste caso, manga-curta será, por fim, o conjunto **[medio]**.

Já no seguinte caso:

```
manga_comprida := [pequeno] * [grande];
```

manga_comprida será o conjunto vazio, **[]**.

Como é constatável, existe o conjunto vazio em Pascal. Ora, isto obriga desde já a retirar uma propriedade da intersecção.

A intersecção entre um conjunto e o conjunto vazio é o próprio conjunto vazio.

UNIÃO (ou REUNIÃO)

A união, ou reunião, de dois ou mais conjuntos resulta num novo conjunto que contenha todos os elementos, sem excepção, de todos os conjuntos envolvidos na operação, sem que haja elementos repetido. Por exemplo, a reunião entre os conjuntos {1, 2, 3, 4} e {0, 3, 4, 6} é o conjunto {0, 1, 2, 3, 4, 6}, e não o conjunto {0, 1, 2, 3, 3, 4, 4, 6}.

Regra geral, na matemática esta operação representa-se como sendo uma **soma**. Em Pascal:

```
manga_curta := [pequeno] + [grande];
```

Neste caso, resulta o conjunto **[pequeno, grande]**.

```
manga_comprida := [pequeno, grande] + [medio, grande];
```

Agora, resulta o conjunto **[pequeno, medio, grande]**.

Mais, uma propriedade da união é intrínseca na análise desta operação:

A união entre um conjunto e o conjunto vazio é o próprio primeiro conjunto.

DIFERENÇA

Pegando em dois conjuntos, a diferença do primeiro pelo segundo resulta num conjunto constituído pelos elementos do primeiro excepto os do segundo. Por exemplo, {1, 2, 3, 4} – {0, 3, 4, 6} dará o conjunto {1, 2}. Já no caso da operação {0, 3, 4, 6} – {1, 2, 3, 4}, o resultado será o conjunto {0, 6}.

A diferença de conjuntos é feita, na matemática, exactamente pela operação da subtracção. Em Pascal, o mesmo acontece:

```
manga_curta := [pequeno, medio] - [pequeno, grande];
```

Neste caso, o conjunto solução é **[medio]**.

```
manga_comprida := [pequeno, medio, grande] - [medio];
```

Já neste caso, o conjunto solução é **[pequeno, grande]**.

Duas propriedades, intimamente relacionadas, podem ser deduzidas:

A diferença de um conjunto pelo conjunto vazio é o próprio primeiro conjunto.

A diferença do conjunto vazio por outro conjunto é o próprio conjunto vazio.

Segue-se um exemplo de atribuição de conjuntos, com operações:

```
manga_curta := [pequeno, medio, grande];  
manga_comprida := manga_curta - [pequeno];  
// manga_comprida = [medio, grande]  
manga_curta := manga_curta * (manga_comprida - [grande]);  
// manga_curta = [medio]
```

1.3. Comparação de conjuntos

Após as operações de conjuntos, segue-se então a **comparação**. Em matemática, isto resume-se em expressões como “pertence a”, “está contido em” ou “não está contido em”. O Pascal tem a capacidade de realizar estas comparações, tendo, para tal, quatro operadores. O resultado resultante destas comparações é sempre e tão-somente um valor **booleano**.

- **= – Igualdade** – ambos os conjuntos contêm os mesmos membros, independentemente da ordem;
- **<> – Desigualdade** – os conjuntos não contêm exactamente os mesmos membros;
- **<= – Inclusão** – cada membro do primeiro conjunto está contido no segundo conjunto;
- **>= – Inclusão** – cada membro do segundo conjunto está contido no primeiro conjunto.

Analise-se, então, uma série de comparações entre conjuntos, e o resultado retornado por cada uma:

- [pequeno, grande] = [pequeno, medio, grande] falso;
- [pequeno, grande] = [grande, pequeno] verdadeiro;
- [pequeno, médio, grande] <> [pequeno..grande] falso;
- [pequeno, grande] <> [grande] verdadeiro;
- [pequeno] <= [pequeno..grande] verdadeiro;
- [pequeno..grande] <= [grande] falso;
- [] <= [pequeno, grande] verdadeiro;
- [pequeno..grande] >= [pequeno, medio] verdadeiro;
- [grande] >= [pequeno, grande] falso.

De novo, conclui-se uma nova propriedade dos conjuntos:

O conjunto vazio está sempre contido em qualquer outro conjunto.

1.4. Análise de uma linha de texto

O programa que se segue é um programa completo, explicado através de comentários ao longo deste. Isto serve para treinar a leitura lógico-formal de um programa em Pascal.

```

program letras_usadas;
(* Programa que determina quais as letras utilizadas numa linha de texto *)
uses crt;
type cartas = set of char;
var usada, naousada : cartas; // letras usadas e não usadas
    contador, caractcontador : 0..80; // contadores
    alfa : char;
    linha : string[80]; // linha de texto, com um máximo de 80 caracteres

procedure ler;
(* Lê uma linha de texto *)
begin
    for contador := 1 to 80 do linha[contador] := ' '; // "limpa" a linha de texto
    writeln('Introduza uma linha de texto');
    readln(linha);
    caractcontador := length(linha);
    {para o contador de caracteres vai o comprimento da string LINHA.}
end;

procedure mostrar;
(* Mostra a análise realizada *)
begin
    writeln;
    write('Letras utilizadas:');
    for alfa := 'A' to 'z' do if [alfa] <= usada then write(' ', alfa);
    // Se o caracter ALPHA está contido no conjunto de letras usadas,
    // então escreve-se esse carácter, uma vez que é uma letra usada na linha de texto.
    writeln;
    writeln;
end;

```

```

begin  (* Bloco principal *)
  ler;  // lê a primeira linha de texto
  while not (upcase(linha) = 'FIM') do begin  // analisa a linha de texto introduzida
    {termina o programa quando de tiver escrito "FIM".}
    usada := [];  // define, no início, que nenhuma letra está utilizada
    nausada := ['A'..'Z', 'a'..'z'];
    // define, no incício, que todas as letras não foram utilizadas
    for contador := 1 to caractcontador do begin
      // analisa a linha de texto, caracter a caracter.
      if [linha[contador]] <= nausada then begin
        {se o conjunto definido pelo caracter nº CONTADOR da linha de texto
        está contido no conjunto das letras não usada, então...}
        usada += [linha[contador]];
        // essa tal letra passa a pertencer ao conjunto das letras usadas...
        nausada -= [linha[contador]];  // e deixa de pertencer às não usadas.
      end;
    end;
    mostrar;  // mostra o resultado
    ler;  // volta a ler nova linha de texto
  end;
end.

```

Propostas de Exercícios

12. Crie um programa que analise uma linha de texto e diga ao utilizador quantas e quais as vogais, bem como as consoantes, utilizadas nessa mesma linha, de forma separada.

Normas:

Devem ser utilizados conjuntos;

Dever-se-á recorrer a um ou mais Record para organizar os vários dados necessários;

Limite de linhas de código (exclusive comentários e linhas em branco): livre.

Desafio 6

Existe um método de determinar os números primos de 2 até n conhecido como *Crivo de Eratosthenes*. O método é o seguinte:

- Gerar uma lista ordenada de números inteiros, de 2 até n ;
- Para um dado número i , da lista, fazê-lo sair da lista e introduzi-lo na de números primos, e, na lista original, eliminar todos os múltiplos de i
- Repetir o passo número 2, para cada número sucessivo de i , começando em $i=2$.

Escreva um programa que utilize este método para determinar os números primos de 2 até n , sendo n um valor dado pelo utilizador. Recorra a conjuntos para resolver este problema.

Normas:

Limite de linhas de código (exclusive comentários e linhas em branco): livre.

2. Estruturação de um programa em Pascal

Muitas vezes, e regra geral, as bases da estruturação de um programa em pascal são dadas no início. Contudo, e pessoalmente falando, eu coloco este método em questão, e deixei-o mais para o fim. Para quê aprender princípios em estruturação se mal se conhece a linguagem e mal se sabe programar em Pascal? Pode-se abordar tal antes de se aprender a linguagem, mas não vai passar de pura teoria. É necessária uma componente prática, penso.

Agora, após as grandes bases sobre Pascal, chega a altura de serem dadas umas noções básicas sobre a boa estruturação de um programa nesta linguagem de programação.

2.1. Estrutura Begin... End

Um procedimento, uma função, uma estrutura de decisão, uma estrutura de repetição e o bloco principal de um programa são todos delimitados pelas palavras `Begin` e `End`. Contudo, é possível criar partes de um programa, estruturando-o em blocos individuais, utilizando estas palavras reservadas. Segue-se um exemplo muito simples.

```
program hello_world;

begin  (* BLOCO PRINCIPAL *)
    begin  (* Bloco de Escrita *)
        writeln('Hello World!');
    end;
    begin  (* Bloco de Pausa *)
        readln;
    end;
end.
```

Ou seja, podem ser criados blocos com uma ou mais linhas de código delimitados por estas duas palavras reservadas.

Estas palavras reservadas não têm de ser obrigatoriamente usadas tão-somente nas situações referidas no início deste subcapítulo.

2.2. A cláusula `Exit`

Este é um **procedimento padrão** do Pascal, que permite que o programa saia de um bloco delimitado por `Begin` e `End`, sem que tenha de executar as restantes linhas de código. Isto **não** se aplica à estrutura `Begin... End` que delimita estruturas de decisão e de repetição, apesar de existirem raros compiladores que aceitem esta cláusula dentro destas estruturas – regra geral, aquando um `Exit` dentro de uma destas estruturas, e que esta esteja no bloco principal, o programa encerra:

```
program hello_world;
var i : integer;

begin  (* BLOCO PRINCIPAL *)
    // ...
    for i := 1 to 10 do begin
        if i = 4 then exit;
        writeln('Hello World!');
    end;
    // ...
end.
```

Devido ao facto de a estrutura de repetição `For` estar dentro do *Bloco Principal*, regra geral, a cláusula `Exit` obriga o programa a sair do bloco principal, ou seja, encerra-o. Contudo, se este ciclo se encontrasse dentro de um “sub-bloco”, então o programa apenas sairia do ciclo `For` e continuava no Bloco Principal:

```
program hello_world;
var i : integer;

begin  (* BLOCO PRINCIPAL *)
    // ...
    begin
        for i := 1 to 10 do begin
            if i = 4 then exit;
            writeln('Hello World!');
        end;
    end;
    writeln('Continuando...');
    // ...
end.
```

Neste caso, quando `i=4`, o programa “saltaria” para fora do ciclo `For` e recomeçava na linha de código `writeln('Continuando...');`.

Veja-se, então, um exemplo muito simples da utilização da cláusula `Exit`:

```
program hello_world;
uses crt;
var i:integer;

procedure escrever;
(* Bloco de Escrita *)
Begin
  if i>=3 then exit;
  writeln('Hello World!');
end;

begin (* BLOCO PRINCIPAL *)
  for i:=1 to 5 do escrever;
  readln;
end.
```

O *output* gerado por este programa é tão-somente o seguinte:



```
Hello World!
Hello World!
```

Isto porque, quando `i >= 3`, é “executada” a cláusula `Exit`, que obriga o programa a “saltar” do procedimento `escrever`, não executando a linha de código seguinte, que é a que faz escrever o texto.

2.3. Etiquetas

Assim que surgiu a programação, surgiu a ideia básica de redireccionamento do programa para certos pontos deste, para que o programa não fosse tão-somente linear, mas sim dinâmico. Existem linguagens de programação, hoje em dia em muito arcaicas, como `FORTRAN` ou `BASIC`, que utilizam abusivamente deste princípio.

Normalmente, em qualquer linguagem de programação, o programa é redireccionado com a palavra reservada `GOTO`, obrigando o programa a dirigir-se a um ponto nomeado (com um nome).

O Pascal também suporta este paradigma. Contudo, devido ao facto de possuir estruturas de repetição, estruturas estas que antigamente não existiam no mundo da programação, a utilização de **etiquetas** torna-se desaconselhada quando a intenção é recorrer a elas de forma abusiva.

Contudo, pode contribuir para parte da boa organização de um programa se utilizada de forma pouca, simples e muito racional. Por exemplo, pode-se criar, num programa que necessite de Menu, uma `Label` (i.e., *etiqueta*) denominada `menu`, e que evite que TODO o programa esteja dentro de uma estrutura `Repeat... Until` ou `While...`

Pegando nesta situação, vamos exemplificar a declaração de uma `Label`, o seu “posicionamento” no programa e a forma de se lhe aceder a certa altura.

```
program exemplo;
uses crt;

label menu;

begin (* bloco principal *)
  // ...
  ;menu: // ponto do programa denominado "menu": é a etiqueta.
  // ...
  goto menu;
  // ...
end.
```

Este é considerado o *método universal* de declaração e utilização das etiquetas, pois, regra geral, funciona em qualquer compilador. Há muitos anos atrás houve breves versões do Pascal que nem sequer possuía a palavra reservada `label`, pois bastava colocar a meio do programa, por exemplo, `Menu` :

Esta forma de declarar etiquetas cumpre melhor o princípio de “linguagem estruturada” que o Pascal segue, pois tudo é declarado no início, incluindo as etiquetas, para que o computador já esteja “preparado” para todas as situações “que derem e vierem”, falando segundo a gíria popular portuguesa.

**NÃO utilize as etiquetas abusivamente.
Dão péssimos hábitos de programação quando utilizadas de tal forma.**

2.4. Características desejáveis num programa

Lista das características **básicas**:

- **Integridade** – precisão dos cálculos;
- **Clareza** – facilidade da leitura do programa (código-fonte);
- **Simplicidade** – o programa deve-se focar no seu objectivo;
- **Eficiência** – velocidade de execução o mais reduzida possível, com resultados exactos, e correcta utilização da memória;
- **Modularidade** – programas complexos podem ser decompostos em módulos, sendo estes procedimentos, funções ou simples blocos `Begin... End`. Deve-se evitar o uso de `Labels` pois podem criar maus vícios de programação, embora sejam igualmente um método.

3. Data e hora do sistema

Antes de dar por terminada esta Parte sobre a linguagem de programação Pascal, é pertinente terminar com uma ferramenta que muitos gostam de utilizar para personalizar os seus programas – mostrar qual a data e a hora do sistema onde o programa é executado.

Para tal, é necessário declarar variáveis do tipo `Word` e utilizar a bibliotecas dos, pois, como o nome da biblioteca indica, a obtenção destes dados implica o acesso ao sistema do computador, que não é mais do que uma função que o DOS realiza constantemente.

```

program relógio_sistema;
uses crt, dos;

var tempo : record
    relógio : record
        hora : word;
        minuto : word;
        segundo : word;
        centesimo_segundo : word;
    end;
    data : record
        ano : word;
        mes : word;
        dia : word;
        dia_semana : word;
    end;
end;

begin
    with tempo do begin
        with relógio do begin
            gettime(hora, minuto, segundo, centesimo_segundo);
            // determina a hora do sistema
        end;
        with data do begin
            getdate(ano, mes, dia, dia_semana);
            // determina a data do sistema
        end
    end

```

```
        end;  
    end;  
    // ...  
end.
```

Este exemplo explica muito bem por si só como funcionam estes comandos, devido à legibilidade das variáveis utilizadas, e organizadas em Record.

Outra ferramenta a saber é a `Delay`, que obriga o programa a esperar um determinado tempo, dito em milissegundos, antes de avançar. Vejamos:

```
// ...  
write('Pausa de 5 segundos... ');  
delay(5000);  
// ...
```

Pode ser útil para a construção de um cronómetro muito simples, ou para evitar que o programa encerre imediatamente assim que lhe é ordenado tal, podendo obrigar a janela ficar em “branco” durante, por exemplo, meio segundo – torna o encerrar do programa mais suave e, possivelmente, agradável.

PARTE VI

Recursividade. Variáveis de Texto Simples.

Estrutura de tentativa. Lista padrão do Pascal.

Esta parte vai dar mais uns “toques” sobre Pascal, abordando a **recursividade**, uma capacidade muito importante do Pascal, terminar as variáveis Text com um exemplo prático sobre **escrita**, **leitura** e **análise** de ficheiros de texto simples, e a **estrutura de tentativa**, extremamente importante para garantir a estabilidade de um programa em processos que são susceptíveis de gerar erros inesperados.

1. Recursividade

Uma das características mais curiosas e impressionantes do Pascal é o facto de as funções e os procedimentos poderem **chamar-se a si mesmos**. Isto é, para realizar um processo ou um cálculo podem-se chamar a si mesmos para auxiliar ou completar tal acção.

O exemplo mais famoso é, sem dúvida, o do cálculo do factorial de um número. Até agora, colocando um problema destes, utilizar-se-ia quase de certeza uma estrutura de repetição For... Downto... para resolver o problema. Contudo, existe uma forma que, até certa medida, é mais intuitiva. Aplicando-se o postulado matemático de que $n! = n * (n-1)!$, verifica-se que pode ser utilizada a capacidade da **recursividade**. Contudo, qualquer estrutura recursiva deve incluir uma **estrutura de finalização**, que mais não é, geralmente, uma **estrutura de decisão** If... Then... Else.... Vejamos a resolução do problema do factorial:

```

program factorial_recursivo;
uses crt;
var numero : integer; // Só um número inteiro positivo "tem" factorial

function factorial(n : integer) : integer;
(* Função Recursiva *)
Begin
  if n<0 then factorial := 0
  // se não é positivo, não há factorial: a função devolve um Integer,
  // logo devolveremos "0".
  else begin
    if n<=1 then factorial := 1
    // Se é 0 ou 1, o factorial é 1 (0! = 1 por postulado matemático)
    else factorial := n * factorial(n-1);
    // Restantes casos, aplica a fórmula n! = n * (n-1)!
  end;
end;

begin (* BLOCO PRINCIPAL *)
  write('Introduza numero inteiro: ');
  readln(numero);
  writeln;
  writeln('O factorial de ', numero, ' e: ', factorial(numero));
  readkey;
end.

```

Caso a função fosse tão-somente

```

function factorial(n : integer) : integer;
begin
  factorial := n * factorial(n-1); // Restantes casos, aplica a fórmula n! = n * (n-1)!
end;

```

O computador entraria num processo infinito, pois não há nada que lhe indique que o factorial é calculado por $n! = n * (n-1) * (n-2) * \dots * 3 * 2 * 1$. O programa iria abaixo quase instantaneamente.

2. Variáveis Text – parte 2

Após terem sido dadas as bases sobre ficheiros de texto simples e outros novos conhecimentos, será criado, para terminar este tipo de dado, dois programas distintos: um cria um ficheiro com dados do utilizador, e outro abre o ficheiro, analisa-o, e diz quais esses dados.

Seja o nome desse ficheiro, criado e analisado, com extensão, `dados.txt`.

O programa que escreve dados no ficheiro é simples, aplicando os conhecimentos adquiridos na Parte IV, somado aos exemplos.

Neste programa, para distinguir algumas variáveis de constantes, as últimas estarão realçadas com maiúsculas.

```

program guarda;
(* Programa que guarda dados num ficheiro *)
var fich : text; // variável TEXT
    dados : record // Registo de dados
        nome : string[40];
        idade : 0..100;
        sexo : char;
        profissao : string[40];
    end;

const nome_fich = 'dados.txt'; // nome do ficheiro
    SEXOS = ['M', 'F']; // dois sexos possíveis
    IDADES = [0..100]; // idade aceites

begin (* BLOCO PRINCIPAL *)
    assign(fich, nome_fich);
    rewrite(fich);
    writeln('Ficheiro aberto. ');
    writeln;
    writeln('Introduza os seguintes dados... ');

    begin (* LEITURA DOS DADOS *)
        with dados do begin
            write(' Nome: '); readln(nome);
            repeat
                write(' Idade (0-100): '); readln(idade);
            until idade in IDADES;
            repeat
                write(' Sexo (M/F): '); readln(sexo);
            until upcase(char(sexo)) in SEXOS;
            write(' Profissao: '); readln(profissao);
        end;
    end;

    begin (* GRAVAÇÃO DOS DADOS *)
        with dados do begin
            writeln(fich, 'DADOS');
            writeln(fich, 'Nome: ', nome);
            writeln(fich, 'Idade: ', idade);
            writeln(fich, 'Sexo: ', sexo);
            writeln(fich, 'Profissao: ', profissao);
        end;
    end;
    close(fich);
    writeln;
    write('Dados gravados. Ficheiro encerrado. ');
    readln;
end.

```

Como se verifica, o programa lê os dados e só de seguida os grava no ficheiro. Para melhor estruturação, estas duas funções diferentes foram inseridas dentro de blocos `Begin... End`.

O programa que **lê** e **analisa** os dados é o mais complicado. O método abordado irá permitir entender como ler o ficheiro e obter dele certas informações de forma **individualizada**. Por exemplo, ao ler no ficheiro `Nome: Thoga31`, poderá, num outro programa, ser de interesse gravar **somente** o nome em si, e não a linha de texto toda. Para tal, o programa que se segue, apesar de não utilizar os dados para nada em especial, vai abordar o método geral que permite esta diferenciação. Isto tornar-se-á útil no caso de se criar um programa com opções e, para que o utilizador não esteja sempre a redefini-las, possas definir de uma vez e ficar gravado num ficheiro que é acedido pelo programa para que este se “personalize” conforme as preferências ditadas pelo utilizador numa anterior utilização.

Antes de apresentar o programa, há que saber a função padrão `eof`: retorna um booleano, e será `True` quando atinge o fim de um ficheiro.

```

program analisa;
(* Programa que analisa os dados do ficheiro *)
uses crt;
var fich : text;
    linha : string; // uma linha do conteúdo do ficheiro
    conteudo : array[1..2] of string; // diferentes conteúdos do ficheiro:
    // e.g.: "NOME: Thoga31" - conteudo[1] = "NOME:"; conteudo[2] = "Thoga31".
    i, j : integer; // contadores

const nome_fich = 'dados.txt';

begin (* BLOCO PRINCIPAL *)
    assign(fich, nome_fich);
    reset(fich);
    writeln('Ficheiro aberto.');
```

(Aqui há um espaço em branco)

```

    while not eof(fich) do begin (* ANALISA CONTEÚDO *)
    (* Enquanto não chega ao fim do ficheiro de texto FICH... *)
        readln(fich, linha); // lê a presente linha.
        conteudo[1] := ''; // no início, não há conteúdo.
        conteudo[2] := ''; // idem.
        j := 1; // começa pelo conteúdo nº1.
        for i:=1 to length(linha) do begin
        // analisa LINHA, caracter a caracter.
            if linha[i]=' ' then j += 1
            // se é espaço, o conteúdo muda (aumenta 1).
            else begin
                if j in [1, 2] then conteudo[j] += linha[i];
                // o array só tem 2 elemento. Logo, se, por erro, j>2, nada é feito;
                // o caracter lido é adicionado a conteudo[j].
            end;
        end;
        if linha <> 'DADOS' then write(' ');
        // faz "tabulação" aos dados em si, não ao título do ficheiro
        writeln(conteudo[1], ' ', conteudo[2]);
    end;

    close(fich);
    writeln;
    write('Ficheiro encerrado.');
```

(Aqui há um espaço em branco)

```

    readln;
end.
```

Neste programa, como o ficheiro a analisar tem no máximo um espaço em cada linha (excepto em possível alteração feita fora do programa), definiu-se um `array` unidimensional com somente dois conteúdos: o conteúdo da linha nº 1 – antes do espaço, e o nº 2 – depois do espaço. Como explicado em comentário, caso o programa leia, por exemplo, `Nome: Thoga31`, ele irá definir que `conteudo[1] = "Nome:"`, que é o texto que está antes do espaço, e `conteudo[2] = "Thoga31"`, que é o texto depois do espaço, e que seria o dado de destaque em toda a linha lida.

O método utilizado é exagerado para este caso particular, pois bastava ler a linha e escrevê-la na janela do programa. Contudo, como explicado, este método pretende mostrar ao mesmo tempo como separar exemplarmente o conteúdo de uma linha para fins já exemplificados e referidos.

3. Sucessor e predecessor

Em muitos tipos de dados pode ser definido o conceito de **sucessor** e **predecessor**

- **Sucessor** – elemento que se segue ao actual;
- **Predecessor** – elemento que antecede o actual.

Por exemplo, o predecessor de "C" é "B", e o sucessor é "D". O predecessor de "6" é "5", e o sucessor é "7". Os comandos que nos dão o sucessor e o predecessor de um elemento são:

```
succ(elemento); // Sucessor  
pred(elemento); // Predecessor
```

4. Estrutura de tentativa – Try... Except... Finally...

Aquando a sua origem, o Pascal não incluía **estruturas de tentativa**. Contudo, com o evoluir do mundo da programação, o Pascal também o teve de fazer, e, hoje em dia, qualquer compilador moderno reconhece e compila esta estrutura.

Teoricamente, esta permite que o programa tente realizar uma operação de forma segura, que, de outra forma, poderia criar um erro inesperado e encerrar o programa instantaneamente. O exemplo típico é o da divisão de um número por zero.

A estrutura de tentativa apresenta este aspecto:

Tentar	Executar Processo
Em erro	Realizar uma série de acções

Caso o processo gira um erro inesperado, a tentativa falha, pois tal não é suportado, pelo que realiza os códigos seguintes. Ou seja, caso as **acções a tentar** falhem, ocorre uma **Excepção** (*exception*).

Em Pascal, ainda é suportado o seguinte: independentemente da tentativa falhar ou não, pode-se "acrescentar" algo para além da futura excepção – para este "acrescimento" existem várias designações a circular. Contudo, uma boa designação será, porventura, **sufixo** – o "acrescimento" acaba por ser um sufixo (mensagem) da tentativa, quer tenha sucesso quer não.

Para utilizar esta estrutura, é necessária uma nova biblioteca: a `sysutils` – utilidades do sistema. A estrutura **básica** é a seguinte:

```
try  
  comandosA;  
except  
  comandosB;  
end;
```

No caso de se querer fazer o "acrescimento" à tentativa:

```
try  
  try  
    comandosA;  
  finally  
    comandosB;  
  end;  
except  
  comandosC;  
end;
```

Este último modelo é considerado por muitos o **modelo padrão**. Contudo, devido ao facto de o sufixo não ser obrigatório, e muitas das vezes importante, a primeira estrutura pode ser considerada o modelo padrão, por ser a mais simples e a que executa as grandes operações-objectivo desta estrutura.

Veja-se, então, a aplicação do exemplo da divisão por zero, onde esta operação se torna **obrigatória**:

```

program tentativa;
uses crt, sysutils;
var i, j, resultado : integer;

begin
  i := 5;
  j := 0; // o denominador vale zero
  write('A tentar dividir, de forma segura, 5 por 0... ');
  try (* ESTRUTURA DE TENTATIVA *)
    resultado := i DIV j;
    write(resultado);
  except (* No caso de excepção *)
    ON E:Exception do begin
      // EXCEPTION é um tipo de dado que indica o tipo de erro ocorrido
      writeln('Falhou');
    end;
  end;
  readln;
end.

```

Por exemplo, para aplicar aqui a estrutura **Finally**, pode-se indicar ao utilizador que a operação se realizou. Desta vez perguntar-se-á os valores da operação ao utilizador:

```

program tentativa;
uses crt, sysutils;
var i, j, resultado : integer;

begin
  write('Introduza numerador: '); readln(i);
  write('Introduza denominador: '); readln(j);
  writeln;
  write('A tentar dividir, de forma segura, ', i, ' por ', j, '... ');

  try (* ESTRUTURA DE TENTATIVA *)
    try // Tentativa com "sufixo"
      resultado := i DIV j;
      write(resultado);
    finally // ... "FINALMENTE" - sufixo
      write(' [Terminado]');
    end;
  except (* EXCEPÇÃO *)
    ON E:Exception do begin
      write(' [Falhou]');
    end;
  end; (* FIM DA ESTRUTURA *)

  writeln;
  readln;
end.

```

Vejam-se dois diferentes *outputs* (os dados introduzidos pelo utilizador encontram-se sublinhados):

```

Introduza numerador: 32
Introduza denominador: 4
A tentar dividir, de forma segura, 32 por 4... 8 [Terminado]

```

```
Introduza numerador: 375
Introduza denominador: 0
```

```
A tentar dividir, de forma segura, 375 por 0... [Terminado][Falhou]
```

A estrutura `Finally` serviu, neste caso, para indicar quando a tentativa terminou. Em caso de exceção, mostra a mensagem adicional "Falhou". Caso tenha sucesso, antes da mensagem "Terminado", é mostrado o resultado da divisão.

Qualquer processo que seja susceptível de ocorrer um erro pode ser encaixado numa estrutura de tentativa, pois evita sistemas complicados de análise e de segurança: quando ocorre o erro, o programa nada mais faz do que, após ter tentado, anunciar a exceção ocorrida.

Alguns procedimentos padrão já têm "imbuído" em si uma espécie de estrutura de tentativa, como é o caso de `val` – o terceiro parâmetro, quando declarado, armazena a localização do primeiro erro que não permitiu a conversão de uma `string` em valor numérico. Este procedimento, por defeito, e quando utilizado o terceiro parâmetro, não necessita de uma estrutura `Try... Except`.

5. Lista padrão do Pascal

Como qualquer linguagem de programação, existe uma **lista padrão** dos procedimentos e das funções padrão e das palavras reservadas.

As listas que se seguem apenas contêm as palavras reservadas, funções e procedimentos padrão abordados até ao momento, pois existe mais.

5.1. Palavras reservadas

As **palavras reservadas** são aquelas que não podem ser redefinidas e fazem os pilares da sintaxe de uma linguagem de programação. Realizam acções e/ou processos bem definidos.

Palavras Reservadas primitivas

AND	ELSE	LABEL	REPEAT	WHILE
ARRAY	END	MOD	SET	WITH
BEGIN	FILE	NOT	STRING	
CASE	FOR	OF	THEN	
CONST	FUNCTION	OR	TO	
DIV	GOTO	PROCEDURE	TYPE	
DO	IF	PROGRAM	UNTIL	
DOWNT0	IN	RECORD	VAR	

Palavras Reservadas modernas

- `EXCEPT`
- `FINALLY`
- `TRY`

5.2. Funções e identificadores padrão

As **funções padrão** são aquelas que já estão incluídas nas bibliotecas: algumas pré-existem mesmo sem qualquer biblioteca definida. Algumas destas funções são, de igual forma, **identificadores**, que são palavras que fazem a identificação, por exemplo, de tipos de dados pré-existentes – por exemplo, `char` é um identificador do tipo de variável carácter, bem como é a função que devolve o carácter de uma certa ordem na tabela ASCII.

Segue-se a lista com as funções e identificadores:

<code>abs</code>	<code>integer</code>	<code>rewrite</code>	<code>trunc</code>
<code>arctan</code>	<code>ln</code>	<code>round</code>	<code>write</code>
<code>boolean</code>	<code>maxint</code>	<code>sin</code>	<code>writeln</code>
<code>char</code>	<code>ord</code>	<code>sqr</code>	
<code>chr*</code>	<code>pred</code>	<code>sqrt</code>	
<code>cos</code>	<code>readln</code>	<code>succ</code>	
<code>eof</code>	<code>real</code>	<code>text</code>	
<code>false</code>	<code>reset</code>	<code>true</code>	

* - Nos compiladores que não suportam `char` como função, utilizam esta função

É de referir que `maxint` é o identificador que dá o valor inteiro (`Integer`) máximo suportado pelo Pascal.

5.3. Procedimentos padrão

O que foi dito sobre as funções padrão aplicam-se aos **procedimentos padrão**.

- `assign`
- `close`
- `read`
- `readln`
- `reset`
- `rewrite`
- `write`
- `writeln`

5.4. Operadores

Os operadores têm uma ordem pela qual são executados. Por exemplo, nas quatro operações básicas, a multiplicação e a divisão são executadas antes que a soma e a subtração.

Segue-se a lista com os operadores ordenados na sua precedência natural, do mais alto (1) para o mais baixo (4):

1. `NOT`
2. `/` `DIV` `MOD` `AND`
3. `+` `-` `OR`
4. `=` `<>` `<` `<=` `>` `>=` `IN`

5.5. Tipos de dados

Um sumário dos tipos de dados abordados até agora:

- `array`
- `boolean`
- `char`
- `integer`
- `real`
- `record`
- `set`
- `string`

5.6. Tabela ASCII

De computador para computador, a tabela ASCII pode sofrer alterações. Contudo, segue-se uma tabela ASCII completa, geralmente sendo designada por **tabela padrão**, a mais comum de se ver.

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Null	32	20	Space	64	40	@	96	60	`	128	80	Ç	160	A0	á	192	C0	Ł	224	E0	α
1	01	Start of heading	33	21	!	65	41	A	97	61	a	129	81	ù	161	A1	í	193	C1	ł	225	E1	β
2	02	Start of text	34	22	"	66	42	B	98	62	b	130	82	é	162	A2	ó	194	C2	ŀ	226	E2	Γ
3	03	End of text	35	23	#	67	43	C	99	63	c	131	83	á	163	A3	ú	195	C3	ŕ	227	E3	π
4	04	End of transmit	36	24	\$	68	44	D	100	64	d	132	84	ä	164	A4	ñ	196	C4	—	228	E4	Σ
5	05	Enquiry	37	25	%	69	45	E	101	65	e	133	85	à	165	A5	Ñ	197	C5	†	229	E5	σ
6	06	Acknowledge	38	26	&	70	46	F	102	66	f	134	86	Å	166	A6	ª	198	C6	‡	230	E6	μ
7	07	Audible bell	39	27	'	71	47	G	103	67	g	135	87	ç	167	A7	º	199	C7	‡	231	E7	τ
8	08	Backspace	40	28	(	72	48	H	104	68	h	136	88	ê	168	A8	¿	200	C8	Ł	232	E8	φ
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i	137	89	ë	169	A9	ƒ	201	C9	ŕ	233	E9	θ
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j	138	8A	è	170	AA	ŕ	202	CA	Ł	234	EA	Ω
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k	139	8B	ı	171	AB	½	203	CB	ŕ	235	EB	δ
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l	140	8C	ı	172	AC	¾	204	CC	ŕ	236	EC	∞
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m	141	8D	ì	173	AD	ı	205	CD	=	237	ED	∞
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n	142	8E	Ë	174	AE	«	206	CE	ŕ	238	EE	ε
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o	143	8F	Ä	175	AF	»	207	CF	Ł	239	EF	∅
16	10	Data link escape	48	30	0	80	50	P	112	70	p	144	90	É	176	BO	☐	208	DO	Ł	240	FO	≡
17	11	Device control 1	49	31	1	81	51	Q	113	71	q	145	91	æ	177	B1	☐	209	D1	ŕ	241	F1	±
18	12	Device control 2	50	32	2	82	52	R	114	72	r	146	92	Æ	178	B2	☐	210	D2	ŕ	242	F2	≥
19	13	Device control 3	51	33	3	83	53	S	115	73	s	147	93	ó	179	B3		211	D3	Ł	243	F3	≤
20	14	Device control 4	52	34	4	84	54	T	116	74	t	148	94	ö	180	B4	†	212	D4	Ł	244	F4	{
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u	149	95	ø	181	B5	‡	213	D5	ŕ	245	F5	}
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v	150	96	ù	182	B6	‡	214	D6	ŕ	246	F6	÷
23	17	End trans. block	55	37	7	87	57	W	119	77	w	151	97	ù	183	B7	ŕ	215	D7	ŕ	247	F7	≈
24	18	Cancel	56	38	8	88	58	X	120	78	x	152	98	ÿ	184	B8	ŕ	216	D8	ŕ	248	F8	•
25	19	End of medium	57	39	9	89	59	Y	121	79	y	153	99	Ö	185	B9	ŕ	217	D9	ŕ	249	F9	•
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z	154	9A	Û	186	BA	ŕ	218	DA	ŕ	250	FA	•
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{	155	9B	ø	187	BB	ŕ	219	DB	☐	251	FB	√
28	1C	File separator	60	3C	<	92	5C	\	124	7C		156	9C	£	188	BC	ŕ	220	DC	☐	252	FC	•
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}	157	9D	¥	189	BD	ŕ	221	DD	☐	253	FD	•
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~	158	9E	€	190	BE	ŕ	222	DE	☐	254	FE	☐
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	□	159	9F	f	191	BF	ŕ	223	DF	☐	255	FF	☐

PARTE VII

Hexadecimais, Binários e Numeração Romana. Message Box.

Apresentam-se nesta parte algumas breves noções introdutórias em questões mais avançadas do Pascal, algumas das quais tocam já procedimentos criados com o Delphi (Object Pascal).

De entre os temas abordados, destaca-se:

- Breve descrição dos Hexadecimais e dos Binários em Pascal e de algumas funções relacionadas;
- Obter a mensagem exacta de um erro (variável **Exception**) numa Estrutura de Tentativa;
- Como alterar o título da consola;
- Obter algumas informações sobre o Disco Rígido.

1. Breve introdução aos Hexadecimais e Binários

O Pascal, de origem, tem capacidade para aceitar a escrita de números inteiros sob a forma hexadecimal e binária. Estando fora do âmbito deste livro a explicação prolongada da representação numérica hexadecimal e binária, explica-se de seguida como introduzir, directamente em código, cada um deste tipo de representação. Considere-se, para tal, a variável **numero**, do tipo **integer** ou **longint**, e iremos gravar o valor **12**.

- Hexadecimal

```
numero := $C;
```

- Binário

```
Numero := %1100;
```

O IDE preparado na íntegra para funções e procedimentos de conversão destas representações é o *Free Pascal*. Declarando-se a unidade **strutils** (utilidades para *strings*), existe uma série de procedimentos e funções. Aqui serão representados tal como estão na unidade, excepto que os parâmetros foram traduzidos para português.

- Converte um número inteiro **valor** numa sequência binária, tendo esta **dígitos** dígitos e onde será acrescentado um espaço a cada **espaços** dígitos. **espaços** é facultativo, e não deverá ser zero.

```
function IntToBin(valor:longint; dígitos:integer;
    espaços:integer):string;
```

- Converte um hexadecimal numa representação numérica dita normal.

```
function Hex2Dec(S : string) : longint;
```

Outras funções e procedimentos existem, mas estão fora do âmbito do presente livro.

2. Estrutura de tentativa – mensagem exacta do erro

Quando abordada a **Estrutura de Tentativa**, recorreu-se logo à adição de uma variável de excepção, **exception**, mas com a qual nada se fez. Ora, qual a vantagem de a declarar?

Imagine-se que se pretende testar um programa complexo e com contornos que possam criar possíveis situações de instabilidade, e o programador pretende saber qual o erro então ocorrido para poder, se possível, corrigi-lo. Contudo, o programa vai abaixo, e a mensagem de erro que a consola lança nem chega a ser vista de tão rápido o processo de erro e encerramento é.

Ora, a variável do tipo **exception** guarda em si o erro ocorrido, erro este que pode ser traduzido por uma mensagem (**message**). Veja-se, então, o exemplo simples da divisão onde pode ocorrer o erro da divisão por zero.

Tradicionalmente, até ao momento, far-se-ia tão-somente o seguinte:

```
try
    resultado := valor1 / valor2;
except
    ON e:exception do begin
        writeln('Ocorreu um erro!');
    end;
end;
```

Podia ser adicionada a mensagem “Divisão por zero”, mas admitamos hipoteticamente que não sabemos o erro que poderá ocorrer.

Então, vamos introduzir o código que nos permite descobrir o erro que ocorre, caso ocorra:

```
try
    resultado := valor1 / valor2;
except
    ON e:exception do begin
        writeln('Ocorreu um erro!');
        writeln('O erro foi o seguinte: ', e.message);
    end;
end;
```

Ou seja, lembrando o paradigma POO, obtém-se a propriedade (atributo) **message** da variável **e**, do tipo **exception**.

A mensagem é debitada em inglês. Veja-se um exemplo de *output* deste excerto de programa, no caso do código que se segue:

```
readln(valor1, valor2);

try
    resultado := valor1 / valor2;
except
    ON e:exception do begin
        writeln('Ocorreu um erro!');
        writeln('O erro foi o seguinte: ', e.message);
    end;
end;
```

(os valores de *input* estão sublinhados)

```
35
0
Ocorreu um erro!
O erro foi o seguinte: Division by zero
```

3. Título da Consola

O título da consola, em *Windows*, por pré-definição é o directório do executável. Por exemplo:

C:\Dev-Pas\exemplo.exe

Claro que o programador gostaria de fazer o seu programa que apresentasse um título na janela da Consola, sem ser uma linha de texto cheia de barras e sem sentido aparente. Neste caso, um bom título seria:

Programa de Exemplo

É possível alterar o título da consola com um procedimento pertencente à unidade **windows**:

```
program exemplo;
uses windows;

begin
  SetConsoleTitle('Programa de Exemplo');
  // etc...
end;
```

4. Message Box

Uma Message Box em Delphi cria-se através de um procedimento que tem quatro parâmetros, existindo dezenas de combinações possíveis. Existem valores pré-definidos, constantes, que, somados, resultam no resultado pretendido. Existem constantes para os botões, o ícone, o botão focado por defeito, etc. Contudo, as Message Boxes serão aqui programadas de forma segura com o auxílio do Editor do *Dev-Pascal*.

4.1. Criação auxiliada no *Dev-Pascal 1.9.2*

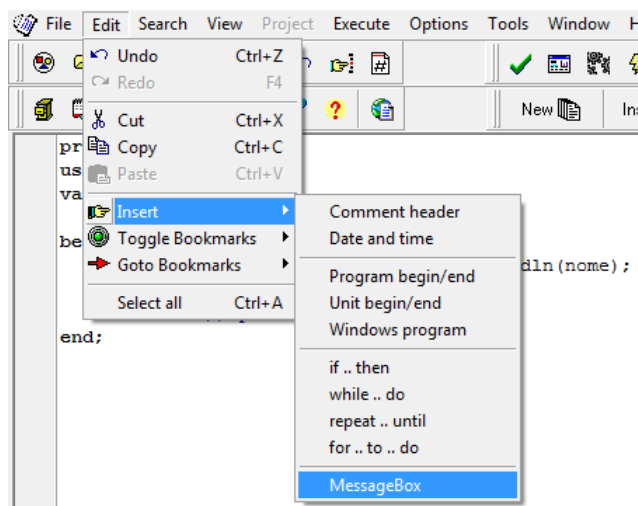
O *Dev-Pascal* possui uma ferramenta que auxilia a criação de uma Message Box, que permite, inclusive, a sua pré-visualização antes de ser aplicada. Por passos:

1. Coloca-se o cursor na zona do código onde se pretende que apareça uma Message Box;

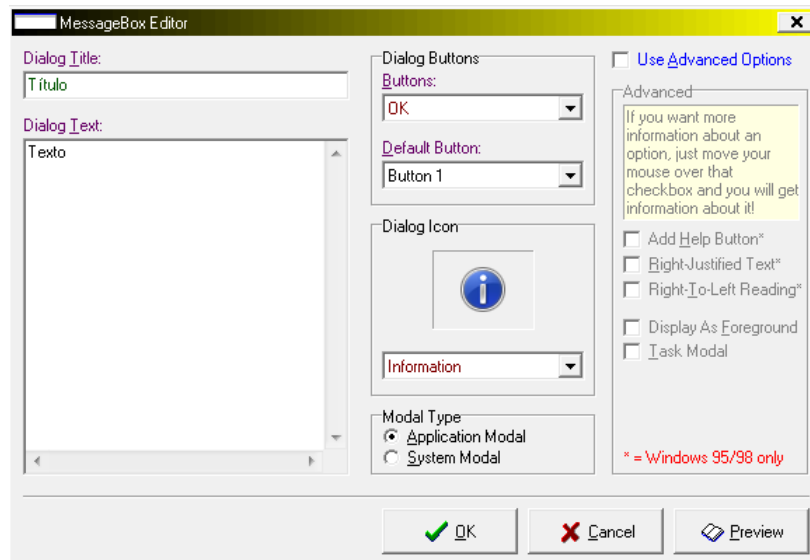
```
program Untitled1;
uses crt;
var nome : string;

begin
  write('Introduza o seu nome: '); readln(nome);
  |
  readln; // pausa
end;
```

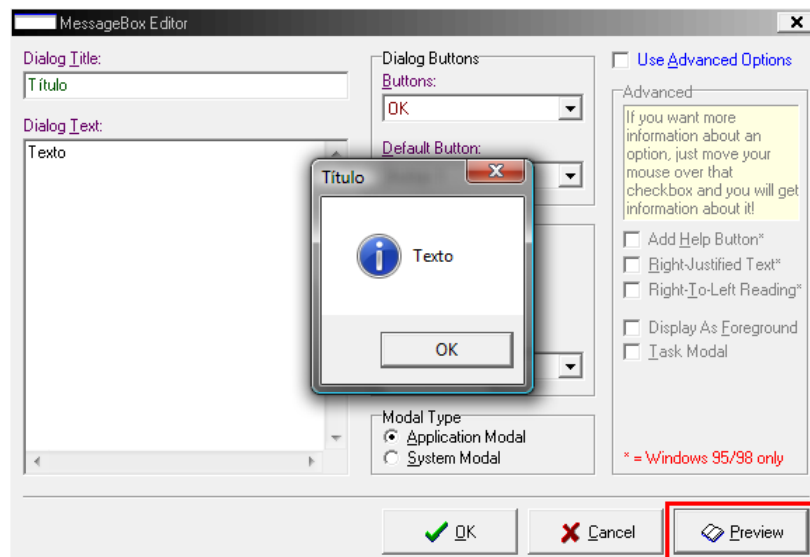
2. Faz-se Edit → Insert → Message Box;



3. Aparece o Editor. Introduce-se o título pretendido, o texto, selecciona-se a sequência de botões desejados, o ícone e o botão que é focado;



4. Para se verificar se a Message Box está como se pretende, faz-se **Preview**.



5. Estando tudo como desejado, faz-se **OK** e o código aparece automaticamente no local onde o cursor foi posicionado no passo 1.

```
program Untitled1;
uses crt;
var nome : string;

begin
  write('Introduza o seu nome: '); readln(nome);
  → MessageBox (0, 'Texto' , 'Titulo', 0 + MB_ICONASTERISK);
  readln; // pausa
end;
```

Neste caso, foi feita apenas uma Message Box modelo para que se pudesse visualizar o código gerado pelo Editor e verificar a posição dos diferentes argumentos.

No caso do programa em questão, adaptar-se-ia do seguinte modo:

```
program Untitled1;
uses crt, windows;
var nome : string;
    msg : string;
    mensagem : pchar;
```



```
begin
  write('Introduza o seu nome: '); readln(nome);
  msg := 'Olá, ' + nome + '!'#0;
  mensagem := @msg[1];
  MessageBox(0, mensagem, 'Mensagem de boas-vindas',
    0 + MB_ICONASTERISK);
  readln; // pausa
end.
```

Está fora do âmbito do presente livro a explicação da conversão de uma **string** para um **pchar**. Fica apenas registado desta forma exemplificativa como o fazer, pois os argumentos de texto da Message Box são do tipo **pchar** e não **string**.

5. Numeração Romana

Declarando a unidade **strings** ou a **strutils**, dependendo do IDE, é possível converter um número inteiro longo (maior que 1 milhão) para numeração romana, e vice-versa. Nenhum erro ocorre, pois são procedimentos seguros.

- Converter de Inteiro (**longint**) para Numeração Romana (**string**)

```
IntToRoman(inteiro, romano);
```

- Converter de Numeração Romana (**string**) para Inteiro (**longint**)

```
RomanToInt(romano, inteiro);
```

Propostas de Exercícios

16. Crie um conversor de numeração romana, enquadrado numa Estrutura de Tentativa, em que, caso um erro ocorra, seja mostrado o erro numa Message Box, em que o título é "Ocorreu um erro" e o texto é tão-somente a mensagem do erro.

Normas:

Limite de número de linhas (excluindo comentários e linhas em branco): 75 linhas.

Notas Finais

Após este Tutorial, o principiante em Pascal deverá ser capaz de criar soluções para os problemas que lhe sejam propostos, recorrendo ao ambiente da consola DOS.

O presente Tutorial **não dispensa** a provável consulta de bibliografia extra.

O Tutorial foi escrito, sempre que possível, na terceira pessoa, excepto em raríssimas excepções. Contudo, tal não indica que eu esteja distante dos seus leitores. Bem pelo contrário, agradeço sugestões e opiniões.

O original deste Tutorial **não** tem a revisão e actualização que esta versão sofreu. Podem ser encontrados, porventura, erros nos programas aí escritos.

Os programas presentes neste Tutorial foram testados e corrigidos. Contudo, poderá ter sempre ocorrido uma falha. Em tal caso, por favor comuniquem-na.

Por favor, as propostas de resolução dos exercícios e desafios devem ser entregues somente por PM (Mensagem Pessoal) no fórum P@P® para serem analisadas primeiramente, e publicadas futuramente por mim, referindo, claro está, o autor da proposta. Várias propostas são aceites, incluindo de exercícios com resolução já colocada: podem existir formas diferentes de abordar o mesmo problema, surgindo soluções distintas.

O Tutorial online pode sofrer alterações sem aviso prévio por parte do autor ou do staff da comunidade Portugal-a-Programar®.

Por fim, gostaria de referir que tive um enorme gosto em elaborar este Tutorial. Tentei ser o mais simples possível, mantendo o máximo de clareza e objectividade possível, com um discurso coeso, tentando não tornar a presente estrutura organizacional do Tutorial “uma desordem”.

O presente Tutorial só pode ser utilizado para áreas e projectos do Portugal-a-Programar® (e não projectos dos seus utilizadores).

O autor,
Igor Nunes
(@thoga31)

Propostas de Resolução de Exercícios e Desafios seleccionados

São apresentadas propostas de resolução a exercícios e desafios seleccionados. Os programas aqui apresentados não incluem comentários, excepto o número de linhas utilizado por cada um.

Exercício 1

```
program ex1; // 33 linhas
uses crt;
var a, b : real;
    oper : string[1];
    valido : boolean;

begin
    writeln('CALCULADORA SIMPLES');
    write('Valor 1: ');
    readln(a);

    repeat
        write('Operador (+, -, /, *): ');
        readln(oper);
    until (oper='+') or (oper='-') or (oper='/') or (oper='*');

    repeat
        write('Valor 2: ');
        readln(b);
        if (oper='/') and (b=0) then begin
            valido := false;
            writeln('ERRO! Divisao por zero!');
        end else valido := true;
    until valido;

    write('Resultado: ');

    if (oper='+') then writeln(a+b:0:10)
    else begin
        if (oper='-') then writeln(a-b:0:10)
        else begin
            if (oper='*') then writeln(a*b:0:10)
            else writeln(a/b:0:10);
        end;
    end;

    write('Prima ENTER para fechar...');
    readln;
end.
```

Exercício 2

```
program ex2; // 18 linhas
uses crt;
var i, alunos, avaliacao, positivas : integer;

begin
    writeln('CONTADOR DE AVALIACOES POSITIVAS');
    write('Numero de alunos? ');
    readln(alunos);

    for i:=1 to alunos do begin
        repeat
            write('Nota do aluno ',i,': ');
```

```

        readln(avaliao);
    until ((avaliao>=0) and (avaliao<=20));
    if (avaliao>=10) then positivas := positivas + 1;
end;

writeln;
writeln('Existem ',positivas,' positivas (',(positivas/alunos)*100:0:1,'%')');
readln;
end.

```

Exercício 3

```

program ex3; // 19 linhas
uses crt;
var numero, maior : real;
    n, i : integer;

begin
    repeat
        write('Quantos numeros vai introduzir? (5-50) ');
        readln(n);
    until (n>=5) and (n<=50);
    maior := 0;
    for i:=1 to n do begin
        write(i,'o numero: ');
        readln(numero);
        if (numero > maior) then maior := numero;
    end;
    writeln;
    writeln('O maior numero foi: ',maior:0:10);
    readln;
end.

```

Exercício 4

```

program ex4; // 24 linhas
uses crt;
var s : string;
    i, j : integer;

begin
    writeln('Este programa tira os espacos a mais de uma STRING');
    writeln;
    write('Escreva uma STRING: ');
    readln(s);
    j := 0;
    for i:=1 to length(s) do
        begin
            j := j+1;
            if (s[i] = ' ') then begin
                s[j] := ' ';
                while (s[i] = ' ') do
                    i := i+1;
                    j := j+1;
                end;
                s[j] := s[i];
            end;
        end;
    writeln('Nova STRING: ',s);
    readln;
end.

```

Exercício 5

```
program ex5; // 24 linhas
uses crt;
var numero : real;
function factorial(n : integer) : integer;
var i, temp : integer;
begin
    if (n=0) or (n=1) then factorial := 1
    else begin
        temp := n;
        for i:=n-1 downto 1 do begin
            temp := temp * i;
        end;
        factorial := temp;
    end;
end;

begin
    repeat
        write('Numero para calcular factorial: ');
        readln(numero);
    until (numero = ROUND(numero)) and (numero>=0);
    writeln;
    writeln('Factorial de ',ROUND(numero),': ',factorial(ROUND(numero)));
    readln;
end.
```

Exercício 6

```
program ex6; // 46 linhas
uses crt;
var a, b : real;
    oper : string[1];

begin
    REPEAT

        textcolor(15);
        writeln('CALCULADORA SIMPLES');
        write('Valor 1: ');
        readln(a);
        write('Operador (+, -, *, /): ');
        readln(oper);
        if (oper='/') then begin
            repeat
                write('Valor 2: ');
                readln(b);
                if (b=0) then begin
                    textcolor(12);
                    writeln('ERRO! Divisao por zero!');
                    textcolor(15);
                end;
            until (b<>0);
        end else begin
            write('Valor 2: ');
            readln(b);
        end;
        writeln;
        write('RESULTADO da operacao: ');
        textcolor(14);
        if (oper='+') then write(a+b:0:10)
        else begin
            if (oper='-') then write(a-b:0:10)
            else begin
                if (oper='*') then write(a*b:0:10)
                else write(a/b:0:10);
            end;
        end;
    end;
```

```

        end;
    end;
    textcolor(15);
    writeln('1 > Repetir operacao');
    writeln('0 > Sair');
    repeat
        readln(oper);
    until (oper='1') or (oper='0');
    if (oper='1') then writeln;

    UNTIL (oper='0');
end.

```

Exercício 7

```

program ex7; // 17 linhas
uses crt;
var i, j : integer;

begin
    for i:=1 to 20 do begin
        for j:=1 to 45 do begin
            case j of
                1..15 : textbackground(1);
                16..30 : textbackground(7);
                31..45 : textbackground(4);
            end;
            write(' ');
        end;
        writeln;
    end;
    readln;
end.

```

Exercício 8

```

program ex8; // 60 linhas
uses crt;
var aluno : array[1..30] of record // RECORD referido no exercício
    nome : string[60];
    matematica : real;
    portugues : real;
end;
media : record // RECORD facultativo: depende do método de cada programador
    matematica : real;
    portugues : real;
end;
quantos, i : integer;

begin
    writeln('ESTATISTICA DA TURMA');
    writeln;
    repeat
        write('Quantos alunos sao? (1-30) ');
        readln(quantos);
    until (quantos>=1) and (quantos<=30);
    writeln;
    for i:=1 to quantos do begin
        writeln('ALUNO ', i);
        with aluno[i] do begin
            write('Nome: ');
            readln(nome);
            repeat
                write('Media a Matematica: ');
                readln(matematica);

```

```

        until (matematica>=0) and (matematica <=20);
        repeat
            write('Media a Portugues: ');
            readln(portugues);
        until (portugues>=0) and (portugues<=20);
    end;
    writeln;
end;
for i:=1 to quantos do begin
    media.matematica += aluno[i].matematica;
    media.portugues += aluno[i].portugues;
end;
with media do begin
    matematica := matematica / quantos;
    portugues := portugues / quantos;
end;
writeln;
writeln('TURMA');
writeln('Media a Portugues: ',media.portugues:0:2);
writeln('Media a Matematica: ',media.matematica:0:2);
writeln;
writeln('ALUNOS APTOS A COMPARECER AO EXAME NACIONAL DE MATEMATICA:');
for i:=1 to quantos do begin
    if (aluno[i].matematica>=9.5) then writeln(' ',aluno[i].nome);
end;
writeln;
writeln('ALUNOS APTOS A COMPARECER AO EXAME NACIONAL DE PORTUGUES:');
for i:=1 to quantos do begin
    if (aluno[i].portugues>=9.5) then writeln(' ',aluno[i].nome);
end;
readln;
end.

```

Exercício 9

```

program ex9; // 48 linhas
uses crt;
var operacao : record
    a : real;
    b : real;
    oper : char;
end;
opcao : char;

begin
    REPEAT

        textbackground(7);
        textcolor(16);
        clrscr;
        writeln('CALCULADORA');
        with operacao do begin
            write(' Valor 1: ');
            readln(a);
            write(' Operacao (+, -, *, /): ');
            repeat
                oper := readkey;
            until (char(oper)='+') or (char(oper)='-') or (char(oper)='*') or (char(oper)='/');
            if (oper='/') then begin
                repeat
                    write(' Valor 2: ');
                    readln(b);
                until (b<>0);
            end else begin
                write(' Valor 2: ');
                readln(b);
            end;
        end;
        write('Resultado: ');
        if (char(oper)='+') then write(a+b:0:10)

```

```
    else begin
        if (char(oper)='-') then write(a-b:0:10)
        else begin
            if (char(oper)='*') then write(a*b:0:10)
            else write(a/b:0:10);
        end;
    end;
end;
writeln;
writeln('1 > Repetir operacao');
writeln('0 > Sair');
repeat
    opcao := readkey;
until (char(opcao)='1') or (char(opcao)='0');
UNTIL (char(opcao)='0');
end.
```

Exercício 11

```
program ex11; // 20 linhas
uses crt;
var face : array[1..6] of integer;
    i : integer;

begin
    randomize;
    for i:=1 to 1000000 do begin
        case round(random(7)) of
            0 : i := i-1; // não há face zero: repete o lançamento
            1 : face[1] += 1;
            2 : face[2] += 1;
            3 : face[3] += 1;
            4 : face[4] += 1;
            5 : face[5] += 1;
            6 : face[6] += 1;
        end;
    end;
    for i:=1 to 6 do writeln('Face ',i,' : ',face[i]);
    readln;
end.
```

Desafio 1

```
program des1; // 21 linhas
uses crt;
var texto : string;

procedure ler(var s : string);
begin
    readln(s);
end;

procedure escrever(s : string);
var i : integer;
begin
    for i:=length(s) downto 1 do write(s[i]);
end;

begin
    write('Escreva linha de texto: ');
    ler(texto);
    writeln;
    write('Texto por ordem inversa: ');
    escrever(texto);
    writeln;
    readln;
end.
```

Desafio 2

```
program des2; // 23 linhas
uses crt;
var i, j : integer;
    key : char;

begin
    writeln('Cores disponiveis: ');
    writeln;
    for i:=1 to 8 do write(i, ' ');
    writeln;
    for i:=1 to 16 do begin
        for j:=1 to 8 do begin
            textcolor(i);
            textbackground(j);
            if (i>=1) and (i<=9) then write('0');
            write(i, ' TESTE ');
        end;
        writeln;
    end;
    textbackground(8);
    textcolor(7);
    writeln;
    readln;
end.
```

Desafio 3

```
program des3; // 16 linhas
uses crt;
var i, j : integer;

begin
  writeln('Tabela ASCII do presente computador:');
  writeln;
  for i:=32 to 255-9 do begin
    write(i, ' ', char(i), ' ', i+1, ' ', char(i+1), ' ', i+2, ' ', char(i+2));
    write(' ', i+3, ' ', char(i+3), ' ', i+4, ' ', char(i+4));
    write(' ', i+5, ' ', char(i+5), ' ', i+6, ' ', char(i+6));
    write(' ', i+7, ' ', char(i+7), ' ', i+8, ' ', char(i+8));
    writeln(' ', i+9, ' ', char(i+9));
    i := i+9;
  end;
  readln;
end.
```

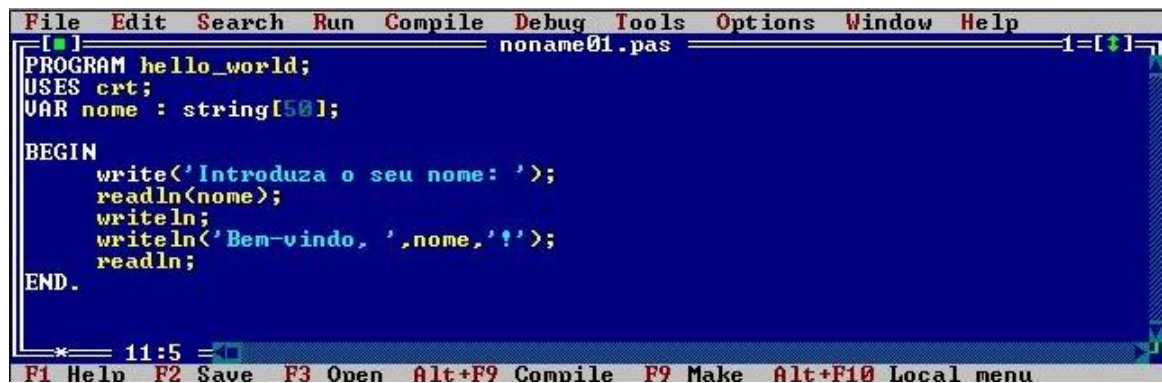
Desafio 4

```
program des4; // 19 linhas
uses crt;

function BolToBin(v : boolean) : integer;
begin
  case v of
    true : BolToBin := 1;
    false : BolToBin := 0;
  end;
end;

function BinToBol(v : integer) : boolean;
begin
  case v of
    1 : BinToBol := true;
    0 : BinToBol := false;
  end;
end;

begin
  // programa
end.
```

The image shows a screenshot of the Free Pascal IDE. The window title is 'noname01.pas'. The menu bar includes File, Edit, Search, Run, Compile, Debug, Tools, Options, Window, and Help. The code editor contains the following Pascal code:

```
PROGRAM hello_world;  
USES crt;  
VAR nome : string[50];  
  
BEGIN  
    write('Introduza o seu nome: ');  
    readln(nome);  
    writeln;  
    writeln('Bem-vindo, ', nome, '!');  
    readln;  
END.
```

The status bar at the bottom shows the time '11:5' and a series of function key shortcuts: F1 Help, F2 Save, F3 Open, Alt+F9 Compile, F9 Make, and Alt+F10 Local menu.

A presente imagem é um exemplo de um programa escrito em Pascal recorrendo ao IDE/compilador *Free Pascal*, versão 2.6.0.

O presente Tutorial foi terminado a 16 de Fevereiro de 2011, antes de ter sido publicado no fórum P@P®.
